Método Numéricos

CAPITULO METODOS ITERATIVOS

Solución de Sistemas de Ecuaciones Lineales

Henry R. Moncada

Universidad Nacional del Callao
Facultad de Ingeniería Mecánica y de Energía

11 de octubre de 2026

Contenido

1. Métodos Iterativos
2. Metodos
   - Método de Jacobi
     - Descomposición matricial
     - Algoritmo
     - Convergencia
     - Ejemplos
     - Implementación en Python
     - Aplicación general
     - Aplicaciones en ingeniería
     - Ventajas y limitaciones
     - Resumen
   - Método de Gauss-Seidel
     - Ejemplos
   - Método de Thomas
     - Fundamento matemático
     - Ejemplos
     - Implementación
     - Aplicación a diferencias finitas
     - Complejidad y estabilidad
   - Método de Relajación Sucesiva (SOR)
     - Ejemplo
     - Python implementación
     - Elección de ω
     - SOR y elementos finitos
   - Gradiente Conjugado
     - Precondicionamiento
     - Código general
   - Generalized Minimal Residual Method (GMRES)
     - Residuo y espacio de Krylov
     - Espacio de Krylov
     - Algoritmo de Arnoldi

¿Por qué necesitamos métodos iterativos?
En el método de elementos finitos, diferencias finitas y otros métodos numéricos es frecuente obtener un sistema algebraico

$$\boxed{\mathbf{Ax} = \mathbf{b}}$$

donde

$$\mathbf{A} \in \mathbb{R}^{n\times n}, \qquad \mathbf{x} \in \mathbb{R}^{n}, \qquad \mathbf{b} \in \mathbb{R}^{n}.$$

Para sistemas pequeños podemos utilizar métodos directos:

Eliminación de Gauss, LU, Cholesky.

Sin embargo, en problemas de ingeniería:

$$n \gg 1,$$

y las matrices suelen ser:

- grandes,
- dispersas,
- simétricas,
- estructuradas por la conectividad de la malla.

Idea fundamental

Los métodos iterativos buscan aproximar la solución mediante una sucesión de vectores

$$\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \ldots$$

hasta alcanzar la precisión requerida.

Métodos iterativos clásicos
Un método iterativo es el desarrollo de una solución aproximada para el sistema de ecuaciones. Para resolver

$$\mathbf{Ax} = \mathbf{b},$$

La aproximación es remplazada sistemáticamente hasta que converge a la respuesta correcta. Algunos métodos iterativos importantes son:

1. Método de Jacobi
2. Método de Gauss - Seidel
3. Método de Relajación Sucesiva (SOR)
4. Método de Gradiente conjugado.
5. Método de GMRES.

¿Preguntas?

Método Iterativos

Sistema de ecuaciones lineales

Muchos problemas de ingeniería conducen a un sistema de ecuaciones lineales:

$$\boxed{\mathbf{Ax} = \mathbf{b}}$$

donde:

$$\mathbf{A} \in \mathbb{R}^{n\times n}, \qquad \mathbf{x} \in \mathbb{R}^{n}, \qquad \mathbf{b} \in \mathbb{R}^{n}.$$

El **método de Jacobi es un método iterativo** utilizado para aproximar la solución de un sistema de ecuaciones lineales.

A diferencia de los métodos directos, como:

Gauss, **Gauss–Jordan**, **factorización LU**

Jacobi genera una sucesión:

$$\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \ldots$$

que, **bajo determinadas condiciones, converge hacia la solución.**

Idea fundamental del método

Supongamos que tenemos:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2, \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n. \end{aligned}$$

Despejamos cada incógnita:

$$x_i = \frac{b_i - \sum_{j\neq i} a_{ij}x_j}{a_{ii}}$$

El método de Jacobi utiliza los valores de la iteración anterior:

$$x_i^{(k+1)} = \frac{b_i - \sum_{j\neq i} a_{ij}x_j^{(k)}}{a_{ii}}$$

Idea clave

Todos los valores de $\mathbf{x}^{(k+1)}$ se calculan utilizando exclusivamente los valores de $\mathbf{x}^{(k)}$.

Descomposición de la matriz $\mathbf{A}$

Escribimos:

$$\boxed{\mathbf{A} = \mathbf{D} + \mathbf{L} + \mathbf{U}}$$

donde:

$$\mathbf{D} = \begin{bmatrix} a_{11} & 0 & 0 \\ 0 & a_{22} & 0 \\ 0 & 0 & a_{33} \end{bmatrix}$$

Matriz diagonal

$$\mathbf{L} = \begin{bmatrix} 0 & 0 & 0 \\ a_{21} & 0 & 0 \\ a_{31} & a_{32} & 0 \end{bmatrix}$$

Triangular inferior

$$\mathbf{U} = \begin{bmatrix} 0 & a_{12} & a_{13} \\ 0 & 0 & a_{23} \\ 0 & 0 & 0 \end{bmatrix}$$

Triangular superior

Por tanto:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \mathbf{D} + \mathbf{L} + \mathbf{U}.$$

Derivación del método de Jacobi

Partimos de:

$$\mathbf{Ax} = \mathbf{b}$$

y utilizamos:

$$\mathbf{A} = \mathbf{D} + \mathbf{L} + \mathbf{U}.$$

Despejando el término diagonal y Premultiplicando por $\mathbf{D}^{-1}$:

$$\begin{aligned}(\mathbf{D} + \mathbf{L} + \mathbf{U})\mathbf{x} &= \mathbf{b} \\ \mathbf{Dx} &= \mathbf{b} - (\mathbf{L} + \mathbf{U})\mathbf{x} \\ \mathbf{x} &= \mathbf{D}^{-1}\mathbf{b} - \mathbf{D}^{-1}(\mathbf{L} + \mathbf{U})\mathbf{x}\end{aligned}$$

Para obtener una fórmula iterativa:

$$\mathbf{x}^{(k+1)} = \mathbf{D}^{-1}\mathbf{b} - \mathbf{D}^{-1}(\mathbf{L} + \mathbf{U})\mathbf{x}^{(k)}$$

o equivalentemente:

$$\mathbf{x}^{(k+1)} = \mathbf{D}^{-1}\left[\mathbf{b} - (\mathbf{L} + \mathbf{U})\mathbf{x}^{(k)}\right]$$

Fórmula componente a componente

La fórmula matricial:

$$\mathbf{x}^{(k+1)} = \mathbf{D}^{-1}\left[\mathbf{b} - (\mathbf{L} + \mathbf{U})\mathbf{x}^{(k)}\right]$$

equivale a:

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{\substack{j=1 \\ j\neq i}}^{n} a_{ij}x_j^{(k)}\right), \text{ para } k = 0, 1, 2, \ldots$$

Para un sistema 3×3:

$$x_1^{(k+1)} = \frac{b_1 - a_{12}x_2^{(k)} - a_{13}x_3^{(k)}}{a_{11}},$$

$$x_2^{(k+1)} = \frac{b_2 - a_{21}x_1^{(k)} - a_{23}x_3^{(k)}}{a_{22}},$$

$$x_3^{(k+1)} = \frac{b_3 - a_{31}x_1^{(k)} - a_{32}x_2^{(k)}}{a_{33}}.$$

Algoritmo del método de Jacobi

1. Definir $\mathbf{A}$, $\mathbf{b}$ y una aproximación inicial $\mathbf{x}^{(0)}$.
2. Seleccionar una tolerancia:
$$\varepsilon_s > 0.$$
3. Para $k = 0, 1, 2, \ldots$ calcular:
$$x_i^{(k+1)} = \frac{b_i - \sum_{j \neq i} a_{ij} x_j^{(k)}}{a_{ii}}.$$
4. Calcular un criterio de error o residuo.
5. Si el criterio de convergencia se cumple:
$$\textbf{PARAR}.$$
6. Si no se cumple:
$$k \leftarrow k + 1$$
y repetir.

Importante

Jacobi calcula todos los componentes de $\mathbf{x}^{(k+1)}$ a partir de $\mathbf{x}^{(k)}$.

Pseudocódigo

Entrada: $\mathbf{A}$, $\mathbf{b}$, $\mathbf{x}^{(0)}$, ε_s, $N_{\text{máx}}$

Salida: $\mathbf{x}^{(k+1)}$

además de:

Proceso

1. $k = 0$
2. Calcular $\mathbf{x}^{(k+1)}$
3. Calcular error
4. Verificar convergencia
5. Repetir

$\mathbf{k}$ = número de iteraciones

y:

$$\mathbf{r} = \|\mathbf{b} - \mathbf{A}\mathbf{x}^{(k+1)}\|$$

como medida del **residuo** ($\mathbf{r}$)

Criterio recomendado

Además del error relativo, es conveniente verificar directamente el residuo del sistema.

Criterios de parada

Una posibilidad es utilizar el error relativo aproximado:

$$e_a^{(k)} = \max_i \left| \frac{x_i^{(k+1)} - x_i^{(k)}}{x_i^{(k+1)}} \right|$$

y detener el proceso cuando:

$$e_a^{(k)} \leq \varepsilon_s$$

También podemos utilizar el **residuo** ($\mathbf{r}^{(k)}$):

$$\boxed{\mathbf{r}^{(k)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}}$$

y:

$$\boxed{\|\mathbf{r}^{(k)}\|_2 \leq \varepsilon}$$

Recomendación

En aplicaciones numéricas es conveniente controlar tanto el cambio entre iteraciones como el residuo.

¿Cuándo converge Jacobi?

El método de Jacobi no converge necesariamente para cualquier matriz. Una condición suficiente muy utilizada es que **A** sea **estrictamente diagonal dominante**:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \qquad \forall i$$

Por ejemplo:

$$\begin{bmatrix} 10 & 2 & 1 \\ 1 & 8 & 2 \\ 2 & 1 & 9 \end{bmatrix}$$

es diagonal dominante porque:

$$\begin{aligned} 10 &> 2 + 1, \\ 8 &> 1 + 2, \\ 9 &> 2 + 1 \end{aligned}$$

También existe un **criterio general basado en el radio espectral de la matriz de iteración**

$$\rho(\mathbf{B_J}) < \mathbf{1}$$

donde:

$$\mathbf{B}_J = -\mathbf{D}^{-1}(\mathbf{L} + \mathbf{U}).$$

Interpretación mediante la matriz de iteración

La fórmula de Jacobi puede escribirse:

$$\mathbf{x}^{(k+1)} = \mathbf{T}_J \mathbf{x}^{(k)} + \mathbf{c}$$

donde:

$$\boxed{\mathbf{T}_J = -\mathbf{D}^{-1}(\mathbf{L} + \mathbf{U})}$$

y:

$$\boxed{\mathbf{c} = \mathbf{D}^{-1}\mathbf{b}.}$$

Por tanto:

$$\mathbf{x}^{(k+1)} = \mathbf{T}_J \mathbf{x}^{(k)} + \mathbf{c}.$$

La convergencia está relacionada con:

$$\rho(\mathbf{T}_J) < 1$$

donde $\rho(\mathbf{T}_J)$ es el **radio espectral:**

$$\rho(\mathbf{T}_J) = \max_i |\lambda_i|$$

¿Cuándo converge Jacobi? El radio espectral

El método de Jacobi puede escribirse como

$$\mathbf{x}^{(k+1)} = \underbrace{\mathbf{T}_J}_{\substack{\text{matriz}\\ \text{de}\\ \text{iteración}}} \mathbf{x}^{(k)} + \mathbf{c},$$

donde

$$\mathbf{T}_J = -\mathbf{D}^{-1}(\mathbf{L}+\mathbf{U}), \qquad \mathbf{c} = \mathbf{D}^{-1}\mathbf{b}.$$

¿Qué ocurre con el error de una iteración a la siguiente?
Sea $\mathbf{x}^*$ la solución exacta y definamos

$$\mathbf{e}^{(k)} = \mathbf{x}^{(k)} - \mathbf{x}^*.$$

Como $\mathbf{x}^* = \mathbf{T}_J\mathbf{x}^* + \mathbf{c}$, al restar ambas expresiones obtenemos

$$\boxed{\mathbf{e}^{(k+1)} = \mathbf{T}_J\mathbf{e}^{(k)}}$$

y, después de k iteraciones,

$$\boxed{\mathbf{e}^{(k)} = \mathbf{T}_J^k\mathbf{e}^{(0)}.}$$

Por tanto, la convergencia depende de cómo actúan las potencias de $\mathbf{T}_J$ sobre el error inicial.

Criterio de convergencia

El método de Jacobi converge para cualquier vector inicial si y solo si

$$\boxed{\rho(\mathbf{T}_J) < 1,}$$

donde el radio espectral es

$$\boxed{\rho(\mathbf{T}_J) = \max_{i=1,\ldots,n} |\lambda_i(\mathbf{T}_J)|}$$

Interpretación del radio espectral

El **radio espectral mide el mayor módulo de los valores propios de la matriz de iteración**. Su valor permite determinar el comportamiento asintótico del error.

Caso 1: $\rho < 1$

$$|\lambda_i| < 1$$

$$\mathbf{T}_J^k \to \mathbf{0}$$

El error tiende a cero.
Jacobi converge.

Caso 2: $\rho > 1$

$$|\lambda_i| > 1 \quad \text{para algún } i$$

Algunas componentes del error pueden crecer.
Jacobi diverge en general.

Caso 3: $\rho = 1$

$$\max_i |\lambda_i| = 1$$

No se garantiza que el error desaparezca.
Hay que analizar más.

¿Por qué aparecen los valores propios?
Si una componente del error está asociada a un vector propio $\mathbf{v}$, entonces

$$\mathbf{T}_J \mathbf{v} = \lambda \mathbf{v}.$$

Después de k iteraciones,

$$\mathbf{T}_J^k \mathbf{v} = \lambda^k \mathbf{v}.$$

Por ello, si $|\lambda| < 1$, esta componente disminuye; si $|\lambda| > 1$, crece.

Importante

El radio espectral describe la convergencia asintótica. No indica por sí solo cuántas iteraciones serán necesarias para alcanzar una tolerancia determinada.

Ejemplo 1: sistema 3×3

Example

Resolver mediante Jacobi:

$$\begin{aligned} 6x_1 + 2x_2 + x_3 &= 22, \\ -x_1 + 8x_2 + 2x_3 &= 30, \\ x_1 - x_2 + 6x_3 &= 23. \end{aligned}$$

En forma matricial:

$$\begin{bmatrix} 6 & 2 & 1 \\ -1 & 8 & 2 \\ 1 & -1 & 6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 22 \\ 30 \\ 23 \end{bmatrix}$$

La solución exacta es:

$$\mathbf{x} = \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix}$$

Solución usando el metodos de jacobi, **(ecuaciones de iteración)**, despejamos cada incógnita:

$$\begin{aligned} x_1 &= \frac{22 - 2x_2 - x_3}{6} \\ x_2 &= \frac{30 + x_1 - 2x_3}{8} \\ x_3 &= \frac{23 - x_1 + x_2}{6} \end{aligned}$$

Por tanto:

$$\begin{aligned} x_1^{(k+1)} &= \frac{22 - 2x_2^{(k)} - x_3^{(k)}}{6} \\ x_2^{(k+1)} &= \frac{30 + x_1^{(k)} - 2x_3^{(k)}}{8} \\ x_3^{(k+1)} &= \frac{23 - x_1^{(k)} + x_2^{(k)}}{6} \end{aligned}$$

iteración inicial

Tomamos:

$$\mathbf{x}^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Para $k = 0$:

$$\left.\begin{aligned} x_1^{(1)} &= \frac{22}{6} = 3{,}6667 \\ x_2^{(1)} &= \frac{30}{8} = 3{,}7500 \\ x_3^{(1)} &= \frac{23}{6} = 3{,}8333 \end{aligned}\right\} \Rightarrow \mathbf{x}^{(1)} = \begin{bmatrix} 3{,}6667 \\ 3{,}7500 \\ 3{,}8333 \end{bmatrix}$$

Ejemplo 1:

segunda iteración

Utilizamos exclusivamente los valores de $\mathbf{x}^{(1)}$:

$$x_1^{(2)} = \frac{22 - 2(3{,}7500) - 3{,}8333}{6} = 1{,}7778$$

$$x_2^{(2)} = \frac{30 + 3{,}6667 - 2(3{,}8333)}{8} = 3{,}2500$$

$$x_3^{(2)} = \frac{23 - 3{,}6667 + 3{,}7500}{6} = 3{,}8472$$

Así:

$$\mathbf{x}^{(2)} = \begin{bmatrix} 1{,}7778 \\ 3{,}2500 \\ 3{,}8472 \end{bmatrix}$$

k	x_1	x_2	x_3
0	0.0000	0.0000	0.0000
1	3.6667	3.7500	3.8333
2	1.7778	3.2500	3.8472
3	1.9421	3.0108	4.0787
4	1.9836	2.9733	4.0114
5	2.0060	2.9957	3.9983
6	2.0017	3.0012	3.9982
7	1.9999	3.0007	4.0000
8	2.0000	3.0000	4.0000

La sucesión converge hacia:

$$\mathbf{x} = \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix}$$

comprobación de la solución

Sustituyendo:

$$\mathbf{x} = \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix}$$

en el sistema:

$$6(2) + 2(3) + 4 = 22,$$
$$-(2) + 8(3) + 2(4) = 30,$$
$$2 - 3 + 6(4) = 23.$$

Por tanto:

$$\mathbf{Ax} = \mathbf{b}$$

y el **residuo r** es:

$$\mathbf{r} = \mathbf{b} - \mathbf{Ax} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

Ejemplo 2: sistema 3×3

Example

Consideremos ahora:

$$\begin{aligned} 10x_1 - 2x_2 + x_3 &= 9, \\ x_1 + 10x_2 - 2x_3 &= 7, \\ 2x_1 - x_2 + 10x_3 &= 6. \end{aligned}$$

En forma matricial:

$$\begin{bmatrix} 10 & -2 & 1 \\ 1 & 10 & -2 \\ 2 & -1 & 10 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 9 \\ 7 \\ 6 \end{bmatrix}$$

Despejamos:

$$\begin{aligned} x_1 &= \frac{9 + 2x_2 - x_3}{10} \\ x_2 &= \frac{7 - x_1 + 2x_3}{10} \\ x_3 &= \frac{6 - 2x_1 + x_2}{10} \end{aligned}$$

primera iteración: partimos de

$$\mathbf{x}^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

Entonces:

$$\left.\begin{aligned} x_1^{(1)} &= \frac{9}{10} = 0{,}9000 \\ x_2^{(1)} &= \frac{7}{10} = 0{,}7000 \\ x_3^{(1)} &= \frac{6}{10} = 0{,}6000 \end{aligned}\right\} \Rightarrow \mathbf{x}^{(1)} = \begin{bmatrix} 0{,}9000 \\ 0{,}7000 \\ 0{,}6000 \end{bmatrix}$$

segunda iteración Utilizamos $\mathbf{x}^{(1)}$:

$$\begin{aligned} x_1^{(2)} &= \frac{9 + 2(0{,}7) - 0{,}6}{10} = 0{,}9800 \\ x_2^{(2)} &= \frac{7 - 0{,}9 + 2(0{,}6)}{10} = 0{,}7300 \\ x_3^{(2)} &= \frac{6 - 2(0{,}9) + 0{,}7}{10} = 0{,}4900 \end{aligned}$$

Por tanto:

$$\mathbf{x}^{(2)} = \begin{bmatrix} 0{,}9800 \\ 0{,}7300 \\ 0{,}4900 \end{bmatrix}$$

Ejemplo 2: iteraciónes

Evolución de las iteraciones

k	x_1	x_2	x_3
0	0.0000	0.0000	0.0000
1	0.9000	0.7000	0.6000
2	0.9800	0.7300	0.4900
3	0.9950	0.7000	0.4930
4	0.9907	0.6991	0.4909
5	0.9907	0.6990	0.4907
6	0.9907	0.6990	0.4907

La solución aproximada es:

$$\mathbf{x} \approx \begin{bmatrix} 0{,}9907 \\ 0{,}6990 \\ 0{,}4907 \end{bmatrix}$$

Este segundo ejemplo permite observar que:

- Jacobi comienza con una aproximación inicial sencilla.
- Cada iteración modifica la aproximación.
- Las aproximaciones se acercan progresivamente a la solución.
- La diagonal dominante favorece la convergencia.

La matriz es estrictamente diagonal dominante:

$$10 > 2 + 1,$$
$$10 > 1 + 2,$$
$$10 > 2 + 1$$

Por tanto, tenemos una condición suficiente para la convergencia.

Ejemplo 1: Jacobi convergente

Considérese el sistema

$$\begin{cases} 4x_1 + x_2 = 9, \\ 2x_1 + 3x_2 = 13 \end{cases}$$

Se desea determinar si el método de Jacobi converge utilizando el radio espectral de su matriz de iteración.

Escribir el sistema matricial
La matriz de coeficientes, el vector de incógnitas y el vector independiente son

$$A = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix}, \qquad \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}, \qquad \mathbf{b} = \begin{pmatrix} 9 \\ 13 \end{pmatrix}.$$

Construir la iteración de Jacobi, despejando cada incógnita

$$x_1^{(k+1)} = \frac{9 - x_2^{(k)}}{4}$$

$$x_2^{(k+1)} = \frac{13 - 2x_1^{(k)}}{3}$$

Por tanto,

$$\begin{pmatrix} x_1^{(k+1)} \\ x_2^{(k+1)} \end{pmatrix} = \underbrace{\begin{pmatrix} 0 & -\frac{1}{4} \\ -\frac{2}{3} & 0 \end{pmatrix}}_{B_J} \begin{pmatrix} x_1^{(k)} \\ x_2^{(k)} \end{pmatrix} + \begin{pmatrix} \frac{9}{4} \\ \frac{13}{3} \end{pmatrix}.$$

Así, la matriz de iteración es

$$B_J = \begin{pmatrix} 0 & -\frac{1}{4} \\ -\frac{2}{3} & 0 \end{pmatrix}$$

Calcular los valores propios

$$\det(\lambda I - B_J) = 0.$$

Sustituyendo la matriz. Calculando el determinante,

$$\begin{vmatrix} \lambda & \frac{1}{4} \\ \frac{2}{3} & \lambda \end{vmatrix} = 0 \Rightarrow \lambda^2 - \frac{1}{4}\frac{2}{3} = \lambda^2 - \frac{1}{6} = 0.$$

Por consiguiente,los dos valores propios son

$$\lambda^2 = \frac{1}{6} \Rightarrow \lambda_1 = \frac{1}{\sqrt{6}}, \quad \lambda_2 = -\frac{1}{\sqrt{6}} \Rightarrow \begin{cases} \lambda_1 \approx 0{,}408248, \\ \lambda_2 \approx -0{,}408248 \end{cases}$$

Determinar el radio espectral

$$\rho(B_J) = \text{máx}\{|\lambda_1|, |\lambda_2|\}. = \text{máx}\left\{\frac{1}{\sqrt{6}}, \frac{1}{\sqrt{6}}\right\}$$

Por tanto, $\rho(B_J) = \dfrac{1}{\sqrt{6}} \approx 0{,}408248 < 1$

Conclusión Como el radio espectral es menor que uno, el método de Jacobi converge para cualquier vector inicial.

Jacobi converge.

En este ejemplo, los errores tienden a cero conforme aumenta el número de iteraciones. La razón es que las potencias de la matriz de iteración tienden a la matriz nula:

$$\lim_{k\to\infty} B_J^k = 0.$$

El sistema tiene como solución exacta

$$x_1 = 1, \qquad x_2 = 5$$

En efecto, $4(1) + 5 = 9$ y $2(1) + 3(5) = 17$. Por tanto, el segundo término independiente debe ser 17 para que la solución indicada sea correcta.

Ejemplo 2: Jacobi no convergente

Considérese ahora el sistema

$$\begin{cases} x_1 + 2x_2 = 5, \\ 2x_1 + x_2 = 4. \end{cases}$$

Determinaremos su radio espectral y evaluaremos la convergencia del método de Jacobi.

Identificar la matriz del sistema
Se tiene

$$A = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}, \qquad \mathbf{b} = \begin{pmatrix} 5 \\ 4 \end{pmatrix}.$$

Obtener las ecuaciones iterativas
Despejando las incógnitas,

$$x_1^{(k+1)} = 5 - 2x_2^{(k)},$$
$$x_2^{(k+1)} = 4 - 2x_1^{(k)}$$

En forma matricial,

$$\begin{pmatrix} x_1^{(k+1)} \\ x_2^{(k+1)} \end{pmatrix} = \underbrace{\begin{pmatrix} 0 & -2 \\ -2 & 0 \end{pmatrix}}_{B_J} \begin{pmatrix} x_1^{(k)} \\ x_2^{(k)} \end{pmatrix} + \begin{pmatrix} 5 \\ 4 \end{pmatrix}.$$

Por tanto,

$$B_J = \begin{pmatrix} 0 & -2 \\ -2 & 0 \end{pmatrix}.$$

Calcular los valores propios

$$\det(\lambda I - B_J) = 0.$$

Es decir, $\begin{vmatrix} \lambda & 2 \\ 2 & \lambda \end{vmatrix} = 0 \Rightarrow \lambda^2 - 4 = 0.$

Factorizando,

$$(\lambda - 2)(\lambda + 2) = 0. \Rightarrow \begin{cases} \lambda_1 = 2, \\ \lambda_2 = -2 \end{cases}$$

Calcular el radio espectral

$$\rho(B_J) = \max\{|\lambda_1|, |\lambda_2|\}. \Rightarrow \rho(B_J) = \max\{2, 2\}.$$

Por consiguiente, $\rho(B_J) = 2 > 1$

Interpretar el resultado: Como el radio espectral es mayor que uno, el método de Jacobi no converge para todo vector inicial.

Jacobi no converge para todo vector inicial.

Para observar el comportamiento, tomemos $\mathbf{x}^{(0)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$. **Se observa que las iteraciones se alejan de la solución en lugar de aproximarse a ella.**

k	$x_1^{(k)}$	$x_2^{(k)}$
0	0	0
1	5	4
2	−3	−6
3	17	10
4	−15	−30

La solución exacta del sistema sí existe y es

$$x_1 = 1, \qquad x_2 = 2$$

Sin embargo, la existencia de una solución no garantiza la convergencia del método de Jacobi.

Comparación de los resultados

Propiedad	Ejemplo 1	Ejemplo 2
$\rho(B_J)$	0,408248	2
Condición	$\rho < 1$	$\rho > 1$
Convergencia	Sí	No para todo inicio

Observaciones importantes

1. El radio espectral se calcula a partir de la matriz de iteración B_J, no directamente de A.
2. Si $\rho(B_J) < 1$, Jacobi converge para cualquier vector inicial.
3. Si $\rho(B_J) > 1$, Jacobi no converge para todo vector inicial; algunos casos particulares pueden comportarse de manera excepcional.
4. La dominancia diagonal estricta por filas es una condición suficiente para la convergencia de Jacobi, pero no es una condición necesaria.
5. La existencia y unicidad de la solución de $A\mathbf{x} = \mathbf{b}$ no garantizan por sí solas que Jacobi converja.

Implementación de Jacobi en Python

La **implementación directa** sigue exactamente la fórmula:

$$x_i^{(k+1)} = \frac{b_i - \sum_{j \neq i} a_{ij} x_j^{(k)}}{a_{ii}}.$$

```
import numpy as np

def jacobi(A, b, x0=None, tol=1e-8, max_iter=100):
    A = np.array(A, dtype=float)
    b = np.array(b, dtype=float)
    n = len(b)

    if x0 is None:
        x = np.zeros(n)
    else:
        x = np.array(x0, dtype=float)

    for k in range(max_iter):
        x_new = np.zeros(n)

        for i in range(n):
            s = 0.0

            for j in range(n):
                if j != i:
                    s += A[i, j] * x[j]

            x_new[i] = (b[i] - s) / A[i, i]

        error = np.linalg.norm(x_new - x, np.inf)
        x = x_new

        if error < tol:
            return x, k + 1

    return x, max_iter
```

```
# This block execute the function jacobi directly

if __name__ == "__main__":

  A = np.array([
      [ 6,  2, 1],
      [-1,  8, 2],
      [ 1, -1, 6]
  ], dtype=float)

  b = np.array([22, 30, 23], dtype=float)

  x, iterations = jacobi(
      A, b,
      x0=[0, 0, 0],
      tol=1e-8,
      max_iter=100
  )

  print("Solucion:")
  print(x)

  print("Iteraciones:")
  print(iterations)

  print("Residuo:")
  print(b - A @ x)
```

Resultado esperado:

$$\mathbf{x} \approx [2,\ 3,\ 4]^T$$

y:

$$\|\mathbf{b} - \mathbf{A}\mathbf{x}\| \approx 0$$

Python: aplicación del Ejemplo 2

```
# This block execute the function jacobi directly
if __name__ == "__main__":

  A = np.array([
      [10, -2,  1],
      [ 1, 10, -2],
      [ 2, -1, 10]
  ], dtype=float)

  b = np.array([9, 7, 6], dtype=float)

  x, iterations = jacobi(
      A, b,
      x0=[0, 0, 0],
      tol=1e-8,
      max_iter=100
  )

  print("Solucion:")
  print(x)

  print("Iteraciones:")
  print(iterations)

  print("Residuo:")
  print(b - A @ x)
```

La solución numérica es aproximadamente:

$$\mathbf{x} \approx [0{,}9907,\ 0{,}6990,\ 0{,}4907]^T$$

Mostrar todas las iteraciones para estudiar el comportamiento del método podemos almacenar cada aproximación:

```
def jacobi_history(A, b, x0=None, tol=1e-8, max_iter=100):
    A = np.array(A, dtype=float)
    b = np.array(b, dtype=float)
    n = len(b)

    if x0 is None:
        x = np.zeros(n)
    else:
        x = np.array(x0, dtype=float)

    history = [x.copy()]

    for k in range(max_iter):
        x_new = np.zeros(n)

        for i in range(n):
            s = np.dot(A[i, :], x) - A[i, i]*x[i]
            x_new[i] = (b[i] - s) / A[i, i]

        history.append(x_new.copy())

        if np.linalg.norm(x_new-x, np.inf) < tol:
            break

        x = x_new
    return np.array(history)

history = jacobi_history(A, b)
print("␣k␣␣␣␣␣␣␣␣x1␣␣␣␣␣␣␣␣␣␣␣x2␣␣␣␣␣␣␣␣␣␣␣x3")
for k, x in enumerate(history):
    print( f"{k:2d}␣␣␣"
           f"{x[0]:10.6f}␣"
           f"{x[1]:10.6f}␣"
           f"{x[2]:10.6f}" )
```

Python: tabla de iteraciones

Esto permite obtener una tabla:

k	x_1	x_2	x_3
0	$\cdots$	$\cdots$	$\cdots$
1	$\cdots$	$\cdots$	$\cdots$
2	$\cdots$	$\cdots$	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$

De esta manera el estudiante puede observar directamente la **evolución de la solución**.

Aplicación general del método

El código anterior puede utilizarse para cualquier sistema:

$$\boxed{\mathbf{Ax} = \mathbf{b}}$$

siempre que:

- $\mathbf{A}$ sea cuadrada.
- $a_{ii} \neq 0$.
- El método sea convergente para el sistema considerado.

Por ejemplo:

```
A = np.array([
    [10,  2,  1],
    [ 1,  8,  2],
    [ 2,  1,  9]
], dtype=float)

b = np.array([13, 17, 20], dtype=float)

x, it = jacobi(A, b)

print("x␣=", x)
print("iteraciones␣=", it)
```

Comparación con la solución directa
Para verificar el resultado podemos utilizar:

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$$

aunque numéricamente es preferible resolver:

$$\mathbf{x} = \text{solve}(\mathbf{A}, \mathbf{b})$$

En Python:

```
x_jacobi, it = jacobi(A, b)

x_directa = np.linalg.solve(A, b)

print("Jacobi:")
print(x_jacobi)

print("Solucion␣directa:")
print(x_directa)

print("Error:")
print(np.linalg.norm(x_jacobi-x_directa))
```

Esto permite comprobar la precisión obtenida mediante el método iterativo.

Cálculo del residuo en Python

Una medida importante de la calidad de la solución es:

$$\boxed{\mathbf{r} = \mathbf{b} - \mathbf{A}\mathbf{x}}$$

En Python:

```
r = b - A @ x

print("Residuo:")
print(r)

print("Norma del residuo:")
print(np.linalg.norm(r))

print("Norma infinito:")
print(np.linalg.norm(r, np.inf))
```

Si:

$$\|\mathbf{r}\| \to 0,$$

entonces la aproximación satisface cada vez mejor el sistema.

Aplicaciones del método de Jacobi

Los métodos iterativos tienen especial interés cuando los sistemas son grandes y dispersos. Algunas aplicaciones:

- Método de elementos finitos.
- Método de diferencias finitas.
- Transferencia de calor.
- Mecánica de fluidos.
- Análisis estructural.
- Problemas de elasticidad.
- Sistemas eléctricos.
- Problemas de difusión.

En sistemas grandes:

$$n \gg 1$$

y con muchas entradas iguales a cero:

$\mathbf{A}$ es una matriz dispersa.

Esto hace atractivos los métodos iterativos.

Jacobi aplicado a diferencias finitas

En una discretización mediante diferencias finitas, la ecuación diferencial puede producir un sistema:

$$\mathbf{AT} = \mathbf{b}$$

donde:

$$\mathbf{T} = \begin{bmatrix} T_1 & T_2 & \cdots & T_n \end{bmatrix}^T$$

contiene las temperaturas desconocidas. Jacobi actualiza:

$$T_i^{(k+1)} = \frac{b_i - \sum_{j \neq i} a_{ij} T_j^{(k)}}{a_{ii}}.$$

Por tanto:

PDE $\rightarrow$ discretización $\rightarrow$ $\mathbf{AT} = \mathbf{b}$ $\rightarrow$ Jacobi

Ventajas del método de Jacobi

Ventajas

- Algoritmo sencillo.
- Fácil de programar.
- No requiere factorización.
- Adecuado para matrices dispersas.
- Fácilmente paralelizable.
- Permite controlar el número de iteraciones.

Desventajas

- Puede converger lentamente.
- No siempre converge.
- Requiere una aproximación inicial.
- Puede necesitar muchas iteraciones.
- Su convergencia depende de la matriz.

Idea

Jacobi sacrifica velocidad por simplicidad y facilidad de implementación.

Jacobi frente a otros métodos

Método	Tipo	Convergencia	Paralelización
Gauss	Directo	–	Baja
Gauss–Jordan	Directo	–	Baja
Jacobi	Iterativo	Condicional	Alta
Gauss–Seidel	Iterativo	Condicional	Media
SOR	Iterativo	Condicional	Media
CG	Iterativo	SPD	Alta
GMRES	Iterativo	General	Moderada

Jacobi es especialmente útil para comprender la estructura fundamental de los métodos iterativos.

Resumen del método de Jacobi

Partimos de: $\mathbf{Ax} = \mathbf{b}$

Descomponemos:

$$\mathbf{A} = \mathbf{D} + \mathbf{L} + \mathbf{U}$$

y obtenemos:

$$\mathbf{x}^{(k+1)} = \mathbf{D}^{-1}\left[\mathbf{b} - (\mathbf{L} + \mathbf{U})\mathbf{x}^{(k)}\right]$$

Componente a componente:

$$x_i^{(k+1)} = \frac{b_i - \sum_{j \neq i} a_{ij} x_j^{(k)}}{a_{ii}}$$

Finalmente verificamos:

$$\|\mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}\| \leq \varepsilon$$

Ideas que debe recordar el estudiante

1. Jacobi es un método **iterativo**.
2. Se necesita una aproximación inicial $\mathbf{x}^{(0)}$.
3. Cada nueva aproximación utiliza únicamente los valores de la iteración anterior.
4. La convergencia no está garantizada para cualquier matriz.
5. La diagonal dominante es una condición suficiente importante.
6. El proceso termina cuando se alcanza una tolerancia.
7. Siempre es conveniente verificar el residuo.

Fórmula fundamental

$$\mathbf{x}^{(k+1)} = \mathbf{D}^{-1}\left[\mathbf{b} - (\mathbf{L} + \mathbf{U})\mathbf{x}^{(k)}\right]$$

¿Preguntas?

Método Jacobi

Método de Gauss-Seidel

El método de Gauss-Seidel es un método iterativo utilizado para resolver sistemas de ecuaciones lineales:

$$\boxed{\mathbf{Ax} = \mathbf{b}}$$

Su principal característica es que los valores recién calculados se utilizan inmediatamente dentro de la misma iteración.

- Es un método iterativo para sistemas de ecuaciones lineales.
- Requiere que $a_{ii} \neq 0$.
- Se despeja cada incógnita respecto de las demás.
- Se propone una aproximación inicial $\mathbf{x}^{(0)}$.
- Los nuevos valores se utilizan inmediatamente.
- El proceso continúa hasta satisfacer un criterio de paro.

Idea fundamental

A diferencia de Jacobi, Gauss-Seidel utiliza inmediatamente $x_i^{(k+1)}$ para calcular $x_{i+1}^{(k+1)}$.

Idea fundamental del método

Consideremos el sistema:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2, \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n. \end{aligned}$$

Despejamos cada incógnita:

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j \right).$$

En Gauss-Seidel:

usar el valor nuevo tan pronto como esté disponible

Por ejemplo:

$$x_1^{(k+1)} \longrightarrow x_2^{(k+1)} \longrightarrow x_3^{(k+1)} \longrightarrow \cdots \longrightarrow x_n^{(k+1)}.$$

Fórmula general de Gauss-Seidel

La fórmula iterativa general es:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left[b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(k)} \right]$$

para:

$$i = 1, 2, \ldots, n, \qquad k = 0, 1, 2, \ldots$$

Observemos la diferencia:

$$\underbrace{x_j^{(k+1)}}_{\text{valor nuevo}} \qquad j < i$$

mientras que:

$$\underbrace{x_j^{(k)}}_{\text{valor anterior}} \qquad j > i.$$

Regla de Gauss-Seidel

Si una componente ya fue calculada durante la iteración actual, se utiliza inmediatamente.

Algoritmo de Gauss-Seidel

1. Escribir el sistema como:
$$\mathbf{Ax} = \mathbf{b}.$$
2. Despejar cada incógnita respecto de las demás.
3. Seleccionar una aproximación inicial:
$$\mathbf{x}^{(0)}.$$
4. Para $k = 0, 1, 2, \ldots$ calcular:
$$x_1^{(k+1)}, x_2^{(k+1)}, \ldots, x_n^{(k+1)}.$$
5. Calcular el error o residuo.
6. Verificar el criterio de paro.
7. Si no se cumple, repetir el proceso.

Inicio → Despeje → Iteración → Error → Converg

Criterio de paro

Un criterio habitual consiste en **utilizar el error relativo aproximado de cada componente**:

$$|\varepsilon_{a,i}| = \left| \frac{x_i^{(k+1)} - x_i^{(k)}}{x_i^{(k+1)}} \right| 100\,\%$$

Se detiene el proceso cuando:

$$|\varepsilon_{a,i}| \leq \varepsilon_s \qquad \forall i$$

donde ε_s **es la tolerancia especificada.**

Otra alternativa, especialmente útil en programación, **es utilizar el residuo:**

$$\boxed{\mathbf{r}^{(k)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}}$$

y verificar:

$$\boxed{\|\mathbf{r}^{(k)}\|_\infty \leq \varepsilon}$$

El número de iteraciones depende de la matriz, del valor inicial y de la tolerancia utilizada.

Gauss-Seidel vs. Jacobi

Método de Jacobi

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$$

Todos los valores nuevos se calculan utilizando únicamente la iteración anterior.

$$\boxed{\mathbf{x}^{(k)} \longrightarrow \mathbf{x}^{(k+1)}}$$

Método de Gauss-Seidel

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left[b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right]$$

Utiliza inmediatamente los valores nuevos.

$$\boxed{x_1^{(k+1)} \rightarrow x_2^{(k+1)} \rightarrow \cdots}$$

Gauss-Seidel suele converger más rápidamente que Jacobi cuando ambos métodos convergen para el mismo problema.

Ejemplo 1: sistema de ecuaciones

Example

Resolver mediante Gauss-Seidel:

$$\begin{aligned} 6x_1 + 2x_2 + x_3 &= 22, \\ -x_1 + 8x_2 + 2x_3 &= 30, \\ x_1 - x_2 + 6x_3 &= 23. \end{aligned}$$

En forma matricial:

$$\begin{bmatrix} 6 & 2 & 1 \\ -1 & 8 & 2 \\ 1 & -1 & 6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 22 \\ 30 \\ 23 \end{bmatrix}$$

Despejamos:

$$\begin{aligned} x_1 &= \frac{22 - 2x_2 - x_3}{6} \\ x_2 &= \frac{30 + x_1 - 2x_3}{8} \\ x_3 &= \frac{23 - x_1 + x_2}{6} \end{aligned}$$

Ecuaciones iterativas: aplicando Gauss-Seidel

$$\begin{aligned} x_1^{(k+1)} &= \frac{22 - 2x_2^{(k)} - x_3^{(k)}}{6} \\ x_2^{(k+1)} &= \frac{30 + x_1^{(k+1)} - 2x_3^{(k)}}{8} \\ x_3^{(k+1)} &= \frac{23 - x_1^{(k+1)} + x_2^{(k+1)}}{6} \end{aligned}$$

Tomamos:

$$\mathbf{x}^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

primera iteración k = 0: (usamos inmediatamente este nuevo valor)

$$\begin{aligned} x_1^{(1)} &= \frac{22 - 2(0) - 0}{6} = \mathbf{3{,}666667} \\ x_2^{(1)} &= \frac{30 + \mathbf{3{,}666667} - 2(0)}{8} = \mathbf{4{,}208333} \\ x_3^{(1)} &= \frac{23 - \mathbf{3{,}666667} + \mathbf{4{,}208333}}{6} = 3{,}923611 \end{aligned}$$

Ejemplo 1: multiples iteraciónes

segunda iteración k = 1

$$x_1^{(2)} = \frac{22 - 2(4{,}208333) - 3{,}923611}{6} = \mathbf{1{,}609954}$$

$$x_2^{(2)} = \frac{30 + \mathbf{1{,}609954} - 2(3{,}923611)}{8} = \mathbf{2{,}970341}$$

$$x_3^{(2)} = \frac{23 - \mathbf{1{,}609954} + \mathbf{2{,}970341}}{6} = 4{,}060065$$

Por tanto: $\mathbf{x}^{(2)} = \begin{bmatrix} 1{,}609954 \\ 2{,}970341 \\ 4{,}060065 \end{bmatrix}$

tercera iteración $k = 2$,

$$x_1^{(3)} = \frac{22 - 2(2{,}970341) - 4{,}060065}{6} = \mathbf{1{,}999875}$$

$$x_2^{(3)} = \frac{30 + \mathbf{1{,}999875} - 2(4{,}060065)}{8} = \mathbf{2{,}984968}$$

$$x_3^{(3)} = \frac{23 - \mathbf{1{,}999875} + \mathbf{2{,}984968}}{6} = 3{,}997515$$

Por tanto: $\mathbf{x}^{(3)} = \begin{bmatrix} 1{,}999875 \\ 2{,}984968 \\ 3{,}997515 \end{bmatrix}$

k	x_1	x_2	x_3	$\lvert\varepsilon_{a,1}\rvert$	$\lvert\varepsilon_{a,2}\rvert$	$\lvert\varepsilon_{a,3}\rvert$
0	0	0	0	–	–	–
1	3.666667	4.208333	3.923611	100.000	100.000	100.000
2	1.609954	2.970341	4.060065	127.750	41.678	3.361
3	1.999875	2.984968	3.997515	19.497	0.490	1.565
4	2.005425	3.001299	3.999312	0.277	0.544	0.045
5	1.999682	3.000132	4.000075	0.287	0.039	0.019
6	1.999943	2.999974	4.000005	0.013	0.005	0.002
7	2.000008	3.000000	3.999999	0.003	0.001	0.0002
8	2.000000	3.000000	4.000000	0.0004	0.00002	0.00003

Para una tolerancia:

$$\varepsilon_s = 0{,}001\,\%$$

la solución aproximada es:

$$\mathbf{x} \approx \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix}$$

La solución exacta del sistema es:

$$\mathbf{x} = \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix}$$

Observamos que:

$$\mathbf{x}^{(1)} = \begin{bmatrix} 3{,}6667 \\ 4{,}2083 \\ 3{,}9236 \end{bmatrix}$$

está relativamente alejada de la solución.

Sin embargo:

$$\mathbf{x}^{(3)} = \begin{bmatrix} 1{,}9999 \\ 2{,}9850 \\ 3{,}9975 \end{bmatrix}$$

ya proporciona una excelente aproximación.

Finalmente:

$$\mathbf{x}^{(8)} \approx \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix}.$$

Gauss-Seidel mejora progresivamente la aproximación utilizando la información calculada durante la propia iteración.

Ejemplo 2: sistema diagonalmente dominante

Example

Consideremos ahora:

$$\begin{aligned} 10x_1 - x_2 + 2x_3 &= 6, \\ -x_1 + 11x_2 - x_3 &= 10, \\ 2x_1 - x_2 + 10x_3 &= 18. \end{aligned}$$

En forma matricial:

$$\begin{bmatrix} 10 & -1 & 2 \\ -1 & 11 & -1 \\ 2 & -1 & 10 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 6 \\ 10 \\ 18 \end{bmatrix}.$$

Despejamos:

$$\begin{aligned} x_1 &= \frac{6 + x_2 - 2x_3}{10} \\ x_2 &= \frac{10 + x_1 + x_3}{11} \\ x_3 &= \frac{18 - 2x_1 + x_2}{10}. \end{aligned}$$

primera iteración

Tomamos:

$$\mathbf{x}^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

Entonces:

$$\begin{aligned} x_1^{(1)} &= \frac{6 + 0 - 2(0)}{10} = \mathbf{0{,}6} \\ x_2^{(1)} &= \frac{10 + \mathbf{0{,}6} + 0}{11} = \mathbf{0{,}963636} \\ x_3^{(1)} &= \frac{18 - 2(\mathbf{0{,}6}) + \mathbf{0{,}963636}}{10} = 1{,}776364 \end{aligned}$$

Por tanto:

$$\mathbf{x}^{(1)} = \begin{bmatrix} 0{,}600000 \\ 0{,}963636 \\ 1{,}776364 \end{bmatrix}$$

Obsérvese que para calcular $x_2^{(1)}$ utilizamos el nuevo valor $x_1^{(1)}$.

Ejemplo 2: algoritmo de cálculo

Para cada iteración:

$$x_1^{(k+1)} = \frac{6 + x_2^{(k)} - 2x_3^{(k)}}{10}$$

$$x_2^{(k+1)} = \frac{10 + x_1^{(k+1)} + x_3^{(k)}}{11}$$

$$x_3^{(k+1)} = \frac{18 - 2x_1^{(k+1)} + x_2^{(k+1)}}{10}$$

La secuencia es:

$$\mathbf{x}^{(0)} \to \mathbf{x}^{(1)} \to \mathbf{x}^{(2)} \to \cdots \to \mathbf{x}^{(k)}.$$

El proceso termina cuando:

$$\max_i |\varepsilon_{a,i}| \leq \varepsilon_s$$

Condición de convergencia

Una condición suficiente muy utilizada es la **dominancia diagonal estricta por filas**

$$|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^{n} |a_{ij}| \quad \textbf{para cada fila } i$$

Para el ejemplo:

$$\begin{bmatrix} 10 & -1 & 2 \\ -1 & 11 & -1 \\ 2 & -1 & 10 \end{bmatrix}$$

tenemos:

$$10 > 1 + 2$$

$$11 > 1 + 1$$

$$10 > 2 + 1.$$

Por tanto:

La matriz es estrictamente diagonal dominante.

Esto garantiza la **convergencia de Gauss-Seidel**.

¿Qué ocurre si no existe dominancia diagonal?

La condición:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|$$

es una **condición suficiente**, no una condición necesaria. Por tanto:

No cumplirla no significa necesariamente que el método diverja.

La convergencia también está relacionada con las propiedades de la matriz. Por ejemplo, Gauss-Seidel converge para matrices:

- estrictamente diagonal dominantes;
- simétricas definidas positivas;
- y otros casos bajo condiciones específicas.

Advertencia

No se debe afirmar que la ausencia de dominancia diagonal implica automáticamente que Gauss-Seidel no converge.

Pseudocódigo de Gauss-Seidel

1. Elegir $\mathbf{x}^{(0)}$.
2. Elegir tolerancia ε_s.
3. Para $k = 0, 1, 2, \ldots$:
   1) Para $i = 1, \ldots, n$:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left[b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right]$$

   2) Calcular el error.
   3) Si el error $\leq \varepsilon_s$, detener.

Devolver $\mathbf{x}^{(k+1)}$

Implementación en Python

```
import numpy as np

def gauss_seidel(A, b, x0=None, tol=1e-6, max_iter=100):
    n = len(b)

    if x0 is None:
        x = np.zeros(n)
    else:
        x = np.array(x0, dtype=float)

    for k in range(max_iter):
        x_old = x.copy()

        for i in range(n):
            suma = 0.0

            for j in range(n):
                if j != i:
                    suma += A[i, j] * x[j]

            x[i] = (b[i] - suma) / A[i, i]

        error = np.max(
            np.abs((x - x_old) / x)
        )

        if error < tol:
            return x, k + 1

    return x, max_iter
```

Python: aplicación al Ejemplo 1

```
# This block executed the function gauss_seidel
if __name__ == "__main__":

  A = np.array([
      [ 6,  2, 1],
      [-1,  8, 2],
      [ 1, -1, 6]
  ], dtype=float)

  b = np.array([22, 30, 23], dtype=float)

  x0 = np.zeros(3)

  x, iterations = gauss_seidel(
    A, b,
    x0=x0,
    tol=1e-6,
    max_iter=100
  )

  print("Solucion:")
  print(x)

  print("Iteraciones:")
  print(iterations)

  print("Residuo:")
  print(b - A @ x)
```

Resultado esperado:

$$\mathbf{x} \approx \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix}$$

Python: aplicación al Ejemplo 2

```
# This block executed the function gauss_seidel
if __name__ == "__main__":

  A = np.array([
      [10, -1,  2],
      [-1, 11, -1],
      [ 2, -1, 10]
  ], dtype=float)

  b = np.array([6, 10, 18], dtype=float)
  x0 = np.zeros(3)
  x, iterations = gauss_seidel(
      A, b,
      x0=x0,
      tol=1e-8,
      max_iter=100
  )
  print("Solucion:")
  print(x)

  print("Iteraciones:")
  print(iterations)

  print("Norma del residuo:")
  print(np.linalg.norm(b - A @ x, np.inf))
```

Podemos almacenar todas las iteraciones:

```
def gauss_seidel_table(A, b, x0, tol=1e-6):

    x = np.array(x0, dtype=float)
    table = []

    for k in range(100):
        old = x.copy()

        for i in range(len(b)):
            s1 = np.dot(A[i,:i], x[:i])
            s2 = np.dot(A[i,i+1:], x[i+1:])

            x[i] = (b[i]-s1-s2)/A[i,i]

        error = np.max(
            np.abs((x-old)/x)
        )

        table.append([k+1, *x, error])
        if error < tol:
            break

    return np.array(table)
```

Esto permite construir automáticamente una tabla como:

$$\mathbf{k}, \quad \mathbf{x_1}, \quad \mathbf{x_2}, \quad \mathbf{x_3}, \quad \varepsilon_\mathbf{a}$$

Para un sistema de dos ecuaciones:

$$a_{11}x_1 + a_{12}x_2 = b_1,$$
$$a_{21}x_1 + a_{22}x_2 = b_2,$$

cada ecuación representa una recta.

La solución corresponde al punto de intersección:

$$\boxed{(x_1, x_2) = \text{solución}}$$

Gauss-Seidel genera una sucesión:

$$\mathbf{x}^{(0)} \to \mathbf{x}^{(1)} \to \mathbf{x}^{(2)} \to \cdots$$

que, cuando converge, se aproxima progresivamente al punto de intersección.

Interpretación: El método no calcula directamente la solución: construye una sucesión de aproximaciones que converge hacia ella.

Ventajas y desventajas

Ventajas

- Fácil de implementar.
- Bajo costo computacional por iteración.
- No requiere invertir explícitamente $\mathbf{A}$.
- Puede ser más rápido que Jacobi.
- Es apropiado para sistemas grandes y dispersos.

Desventajas

- No siempre converge.
- Puede converger lentamente.
- Depende del orden de las ecuaciones.
- Requiere una diagonal adecuada.
- La elección de $\mathbf{x}^{(0)}$ puede influir en el número de iteraciones.

Aplicaciones en ingeniería

Gauss-Seidel puede utilizarse para resolver los sistemas lineales que aparecen en diferentes problemas de ingeniería:

- Método de diferencias finitas.
- Transferencia de calor.
- Mecánica de fluidos.
- Análisis estructural.
- Elementos finitos.
- Problemas de potencial.
- Sistemas eléctricos.
- Problemas de difusión.

En particular, después de discretizar una ecuación diferencial, frecuentemente se obtiene:

$$\mathbf{KQ} = \mathbf{F}$$

donde $\mathbf{K}$ es una matriz de rigidez y $\mathbf{Q}$ contiene las incógnitas nodales.

Gauss-Seidel aplicado a elementos finitos

En elementos finitos:

$$\mathbf{KQ} = \mathbf{F}$$

puede representar, por ejemplo:

$$\mathbf{K} = \begin{bmatrix} k_{11} & k_{12} & \cdots \\ k_{21} & k_{22} & \cdots \\ \vdots & \vdots & \ddots \\ \vdots & \vdots & \ddots \end{bmatrix} \quad \mathbf{y} \quad \mathbf{Q} = \begin{bmatrix} Q_1 \\ Q_2 \\ \vdots \\ Q_n \end{bmatrix}.$$

Gauss-Seidel permite obtener:

$$Q_1^{(k+1)} \to Q_2^{(k+1)} \to \cdots \to Q_n^{(k+1)}.$$

Aplicación: Para sistemas FEM grandes y dispersos, los métodos iterativos pueden evitar operaciones directas costosas.

Resumen del método

1. Partimos del sistema:
$$\mathbf{Ax} = \mathbf{b}.$$
2. Despejamos cada incógnita.
3. Elegimos:
$$\mathbf{x}^{(0)}.$$
4. Calculamos secuencialmente:
$$x_1^{(k+1)}, x_2^{(k+1)}, \ldots, x_n^{(k+1)}.$$
5. Utilizamos inmediatamente los valores nuevos.
6. Calculamos el error o residuo.
7. Repetimos hasta satisfacer la tolerancia.

$$\boxed{\mathbf{x}^{(0)} \to \mathbf{x}^{(1)} \to \mathbf{x}^{(2)} \to \cdots \to \mathbf{x}}$$

Resumen: la idea que debe recordar el estudiante

Jacobi

Todos los cálculos utilizan valores de la iteración anterior

Gauss-Seidel

Cada valor nuevo se utiliza inmediatamente

$$\boxed{x_1^{(k+1)} \to x_2^{(k+1)} \to x_3^{(k+1)} \to \cdots}$$

Mensaje principal

Gauss-Seidel aprovecha inmediatamente la información disponible para acelerar, en muchos casos, la convergencia.

¿Preguntas?

Método Gauss-Seidel

Método de Thomas

El **método de Thomas** es un algoritmo especializado para resolver sistemas de ecuaciones lineales cuya matriz de coeficientes es **tridiagonal**.
Un sistema tridiagonal puede escribirse como

$$\mathbf{Ax} = \mathbf{d},$$

donde

$$\mathbf{A} = \begin{bmatrix} b_1 & c_1 & 0 & \cdots & 0 \\ a_2 & b_2 & c_2 & \ddots & \vdots \\ 0 & a_3 & b_3 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & c_{n-1} \\ 0 & \cdots & 0 & a_n & b_n \end{bmatrix}.$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \qquad \mathbf{d} = \begin{bmatrix} d_1 \\ d_2 \\ \vdots \\ d_n \end{bmatrix}.$$

Estructura del sistema tridiagonal

El sistema tiene la forma

$$\begin{aligned} b_1 x_1 + c_1 x_2 &= d_1, \\ a_2 x_1 + b_2 x_2 + c_2 x_3 &= d_2, \\ a_3 x_2 + b_3 x_3 + c_3 x_4 &= d_3, \\ &\vdots \\ a_n x_{n-1} + b_n x_n &= d_n. \end{aligned}$$

Tres diagonales

Superdiagonal
c_i

Diagonal principal
b_i

Subdiagonal
a_i

Características

- La matriz es dispersa.
- Solo existen tres diagonales no nulas.
- No es necesario almacenar todos los elementos de la matriz.
- El método requiere aproximadamente $O(n)$ operaciones.
- Es mucho más eficiente que aplicar una eliminación gaussiana general.

¿Dónde aparecen los sistemas tridiagonales?

Los sistemas tridiagonales aparecen frecuentemente al discretizar ecuaciones diferenciales mediante **diferencias finitas**. Por ejemplo, para la ecuación de difusión unidimensional

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2},$$

una discretización espacial mediante diferencias centrales conduce a

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i-1} - 2u_i + u_{i+1}}{\Delta x^2}.$$

Al utilizar un esquema implícito se obtiene un sistema del tipo

$$a_i u_{i-1}^{n+1} + b_i u_i^{n+1} + c_i u_{i+1}^{n+1} = d_i.$$

Idea fundamental

Cada ecuación solamente relaciona una incógnita con sus dos vecinas. Por esta razón, la matriz resultante es tridiagonal.

Ventaja computacional

Una matriz general de dimensión $n \times n$ contiene

$$n^2$$

elementos.
Una matriz tridiagonal solamente necesita almacenar

$$3n - 2$$

elementos.

Matriz general

$$\mathbf{A} = \begin{bmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{bmatrix}$$

Almacenamiento:

$$O(n^2)$$

Matriz tridiagonal

$$\mathbf{A} = \begin{bmatrix} * & * & 0 & 0 \\ * & * & * & 0 \\ 0 & * & * & * \\ 0 & 0 & * & * \end{bmatrix}$$

Almacenamiento:

$$O(n)$$

Ventaja

El método de Thomas aprovecha directamente la estructura tridiagonal, reduciendo significativamente el costo computacional y el almacenamiento.

Idea del método de Thomas

El método de Thomas es una versión especializada de la **eliminación de Gauss** para sistemas tridiagonales.

Se divide en dos etapas:

1. **Eliminación hacia adelante**

$$a_i \to 0.$$

2. **Sustitución hacia atrás**

$$x_n, x_{n-1}, \ldots, x_1.$$

Durante la eliminación se modifican únicamente los coeficientes necesarios de las tres diagonales.

Idea

No necesitamos transformar toda la matriz como en la eliminación de Gauss convencional. Solamente actualizamos los coeficientes que intervienen en la estructura tridiagonal.

Eliminación hacia adelante

Partimos del sistema

$$b_1 x_1 + c_1 x_2 = d_1.$$

Dividiendo la primera ecuación por b_1:

$$x_1 + \frac{c_1}{b_1} x_2 = \frac{d_1}{b_1}.$$

Definimos

$$w_1 = \frac{c_1}{b_1}, \qquad g_1 = \frac{d_1}{b_1}.$$

Por tanto,

$$x_1 + w_1 x_2 = g_1.$$

Para la segunda ecuación:

$$a_2 x_1 + b_2 x_2 + c_2 x_3 = d_2.$$

Sustituyendo

$$x_1 = g_1 - w_1 x_2,$$

se obtiene

$$(b_2 - a_2 w_1) x_2 + c_2 x_3 = d_2 - a_2 g_1.$$

Dividiendo por

$$b_2 - a_2 w_1,$$

obtenemos

$$x_2 + w_2 x_3 = g_2,$$

donde

$$w_2 = \frac{c_2}{b_2 - a_2 w_1}$$
$$g_2 = \frac{d_2 - a_2 g_1}{b_2 - a_2 w_1}$$

De manera general:

$$w_i = \frac{c_i}{b_i - a_i w_{i-1}}$$
$$g_i = \frac{d_i - a_i g_{i-1}}{b_i - a_i w_{i-1}}$$

Para

$$i = 2, 3, \ldots, n-1.$$

Algoritmo de Thomas

Paso 1: primera ecuación

$$w_1 = \frac{c_1}{b_1}, \qquad g_1 = \frac{d_1}{b_1}.$$

Paso 2: eliminación hacia adelante

$$\left.\begin{aligned} w_i &= \frac{c_i}{b_i - a_i w_{i-1}}, \\ g_i &= \frac{d_i - a_i g_{i-1}}{b_i - a_i w_{i-1}}. \end{aligned}\right\} \text{ para } i = 2, \ldots, n-1$$

Paso 3: última ecuación

$$g_n = \frac{d_n - a_n g_{n-1}}{b_n - a_n w_{n-1}}$$
$$x_n = g_n.$$

Sustitución hacia atrás

Una vez obtenidos los coeficientes g_i y w_i, comenzamos con la última incógnita:

$$\boxed{x_n = g_n}.$$

Luego retrocedemos:

$$\boxed{x_i = g_i - w_i x_{i+1}}$$

para

$$i = n-1, n-2, \ldots, 2, 1.$$

Por ejemplo:

$$\begin{aligned} x_{n-1} &= g_{n-1} - w_{n-1} x_n, \\ x_{n-2} &= g_{n-2} - w_{n-2} x_{n-1}, \\ &\vdots \\ x_1 &= g_1 - w_1 x_2 \end{aligned}$$

Resultado

Al finalizar obtenemos el vector solución

$$\mathbf{x} = [x_1, x_2, \ldots, x_n]^T.$$

Algoritmo completo

1. Definir los vectores
$$\mathbf{a}, \quad \mathbf{b}, \quad \mathbf{c}, \quad \mathbf{d}.$$
2. Calcular
$$w_1 = \frac{c_1}{b_1}, \qquad g_1 = \frac{d_1}{b_1}.$$
3. Para $i = 2, \ldots, n-1$, calcular
$$\begin{aligned} m_i &= b_i - a_i w_{i-1}, \\ w_i &= \frac{c_i}{m_i}, \\ g_i &= \frac{d_i - a_i g_{i-1}}{m_i}. \end{aligned}$$
4. Calcular
$$g_n = \frac{d_n - a_n g_{n-1}}{b_n - a_n w_{n-1}}.$$
5. Obtener
$$x_n = g_n.$$
6. Realizar sustitución hacia atrás:
$$x_i = g_i - w_i x_{i+1}.$$

Pseudocódigo

1. Entrada:
$$a, b, c, d, n.$$
2.
$$w_1 = \frac{c_1}{b_1} \qquad g_1 = \frac{d_1}{b_1}$$
3. Para $i = 2, \ldots, n-1$:
$$m = b_i - a_i w_{i-1}$$
$$w_i = \frac{c_i}{m}$$
$$g_i = \frac{(d_i - a_i g_{i-1})}{m}$$
4.
$$g_n = \frac{d_n - a_n g_{n-1}}{b_n - a_n w_{n-1}}.$$
5.
$$x_n = g_n.$$
6. Para $i = n-1, \ldots, 1$:
$$x_i = g_i - w_i x_{i+1}.$$
7. Salida:
$$\boxed{\mathbf{x}}$$

Ejemplo 1: sistema tridiagonal

Resolver mediante el método de Thomas:

$$\begin{aligned} 4x_1 - x_2 &= 3, \\ -x_1 + 4x_2 - x_3 &= 4, \\ -x_2 + 4x_3 - x_4 &= 5, \\ -x_3 + 4x_4 &= 5. \end{aligned}$$

La matriz es

$$\mathbf{A} = \begin{bmatrix} 4 & -1 & 0 & 0 \\ -1 & 4 & -1 & 0 \\ 0 & -1 & 4 & -1 \\ 0 & 0 & -1 & 4 \end{bmatrix}.$$

Los vectores son

$$\mathbf{a} = \begin{bmatrix} 0 \\ -1 \\ -1 \\ -1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 4 \\ 4 \\ 4 \\ 4 \end{bmatrix},$$

$$\mathbf{c} = \begin{bmatrix} -1 \\ -1 \\ -1 \\ 0 \end{bmatrix}, \quad \mathbf{d} = \begin{bmatrix} 3 \\ 4 \\ 5 \\ 5 \end{bmatrix}.$$

eliminación hacia adelante

Para $i = 1$:

$$w_1 = \frac{-1}{4} = -0{,}25, \qquad g_1 = \frac{3}{4} = 0{,}75.$$

Para $i = 2$:

$$m_2 = 4 - (-1)(-0{,}25) = 3{,}75,$$

$$w_2 = \frac{-1}{3{,}75} = -0{,}266667,$$

$$g_2 = \frac{4 - (-1)(0{,}75)}{3{,}75} = 1{,}266667.$$

Para $i = 3$:

$$m_3 = 4 - (-1)(-0{,}266667) = 3{,}733333,$$

$$w_3 = \frac{-1}{3{,}733333} = -0{,}267857,$$

$$g_3 = \frac{5 - (-1)(1{,}266667)}{3{,}733333} = 1{,}678571.$$

Para $i = 4$:

$$m_4 = 4 - (-1)(-0{,}267857) = 3{,}732143$$

$$g_4 = \frac{5 - (-1)(1{,}678571)}{3{,}732143} = 1{,}791388$$

y

$$\boxed{x_4 = g_4 = 1{,}791388}.$$

Ahora comenzamos la sustitución hacia atrás.

Ejemplo 1: sustitución hacia atrás

Tenemos

$$x_4 = 1{,}791388.$$

Entonces

$$x_3 = g_3 - w_3 x_4$$
$$x_3 = 1{,}678571 - (-0{,}267857)(1{,}791388) = 2{,}158333$$

Luego

$$x_2 = g_2 - w_2 x_3$$
$$x_2 = 1{,}266667 - (-0{,}266667)(2{,}158333) = 1{,}842222$$

Finalmente,

$$x_1 = g_1 - w_1 x_2$$
$$x_1 = 0{,}75 - (-0{,}25)(1{,}842222) = 1{,}210556$$

La solución aproximada obtenida con el método de Thomas es

$$\mathbf{x} \approx \begin{bmatrix} 1{,}210556 \\ 1{,}842222 \\ 2{,}158333 \\ 1{,}791388 \end{bmatrix}$$

Verificación mediante el residuo:

$$\mathbf{r} = \mathbf{d} - \mathbf{A}\mathbf{x}.$$

Para una implementación computacional podemos calcular

$$\|\mathbf{r}\|_\infty = \max_i |r_i|$$

Importante

El método de Thomas no es iterativo. Una vez realizadas la eliminación hacia adelante y la sustitución hacia atrás, se obtiene directamente la solución.

Ejemplo 1: Python

```
import numpy as np

a = np.array([0, -1, -1, -1], dtype=float)
b = np.array([4,  4,  4,  4], dtype=float)
c = np.array([-1, -1, -1, 0], dtype=float)
d = np.array([3, 4, 5, 5], dtype=float)

n = len(d)
w = np.zeros(n)
g = np.zeros(n)
x = np.zeros(n)

w[0] = c[0] / b[0]
g[0] = d[0] / b[0]

for i in range(1, n):
    m = b[i] - a[i] * w[i-1]

    if i < n-1:
        w[i] = c[i] / m

    g[i] = (d[i] - a[i] * g[i-1]) / m

x[-1] = g[-1]
for i in range(n-2, -1, -1):
    x[i] = g[i] - w[i] * x[i+1]

print("Solucion:")
print(x)
```

Ejemplo 2: sistema tridiagonal

Resolver:

$$\begin{aligned} 3x_1 + x_2 &= 7, \\ x_1 + 4x_2 + x_3 &= 8, \\ x_2 + 5x_3 + x_4 &= 14, \\ x_3 + 3x_4 &= 10. \end{aligned}$$

Por tanto,

$$\mathbf{A} = \begin{bmatrix} 3 & 1 & 0 & 0 \\ 1 & 4 & 1 & 0 \\ 0 & 1 & 5 & 1 \\ 0 & 0 & 1 & 3 \end{bmatrix}.$$

Los vectores son

$$\begin{aligned} a &= [0, 1, 1, 1]^T, \\ b &= [3, 4, 5, 3]^T, \\ c &= [1, 1, 1, 0]^T, \\ d &= [7, 8, 14, 10]^T \end{aligned}$$

eliminación hacia adelante

Primera ecuación, para $i = 1$:

$$\begin{aligned} w_1 &= \frac{1}{3} = 0{,}333333, \\ g_1 &= \frac{7}{3} = 2{,}333333. \end{aligned}$$

Para $i = 2$:

$$\begin{aligned} m_2 &= 4 - (1)(0{,}333333) = 3{,}666667, \\ w_2 &= \frac{1}{3{,}666667} = 0{,}272727, \\ g_2 &= \frac{8 - (1)(2{,}333333)}{3{,}666667} = 1{,}545455 \end{aligned}$$

Para $i = 3$:

$$\begin{aligned} m_3 &= 5 - (1)(0{,}272727) = 4{,}727273, \\ w_3 &= \frac{1}{4{,}727273} = 0{,}211538, \\ g_3 &= \frac{14 - 1{,}545455}{4{,}727273} = 2{,}634615 \end{aligned}$$

Para $i = 4$:

$$\begin{aligned} m_4 &= 3 - (1)(0{,}211538) = 2{,}788462 \\ g_4 &= \frac{10 - (1)(2{,}634615)}{2{,}788462} = 2{,}641026 \end{aligned}$$

y $\boxed{x_4 = 2{,}641026}$.

Ejemplo 2: sustitución hacia atrás

$$x_4 = 2{,}641026.$$

Luego,

$$x_3 = g_3 - w_3 x_4$$
$$x_3 = 2{,}634615 - (0{,}211538)(2{,}641026) = 2{,}075214$$

Después,

$$x_2 = 1{,}545455 - (0{,}272727)(2{,}075214) = 0{,}979942$$

Finalmente,

$$x_1 = 2{,}333333 - (0{,}333333)(0{,}979942), 1 = 2{,}006686$$

La solución aproximada es

$$\mathbf{x} \approx \begin{bmatrix} 2{,}006686 \\ 0{,}979942 \\ 2{,}075214 \\ 2{,}641026 \end{bmatrix}$$

La solución puede verificarse sustituyendo los valores obtenidos en el sistema original.

$$\mathbf{r} = \mathbf{d} - \mathbf{A}\mathbf{x}.$$

Una implementación computacional también permite comparar el resultado con una solución obtenida mediante

`numpy.linalg.solve(A,d)`.

Ejemplo 2: Python

```
import numpy as np

a = np.array([0, 1, 1, 1], dtype=float)
b = np.array([3, 4, 5, 3], dtype=float)
c = np.array([1, 1, 1, 0], dtype=float)
d = np.array([7, 8, 14, 10], dtype=float)

n = len(d)

w = np.zeros(n)
g = np.zeros(n)
x = np.zeros(n)

w[0] = c[0] / b[0]
g[0] = d[0] / b[0]

for i in range(1, n):
    m = b[i] - a[i] * w[i-1]

    if i < n-1:
        w[i] = c[i] / m

    g[i] = (d[i] - a[i] * g[i-1]) / m

x[-1] = g[-1]

for i in range(n-2, -1, -1):
    x[i] = g[i] - w[i] * x[i+1]

print("Solucion:")
print(x)
```

Implementación general en Python

```
import numpy as np

def thomas(a, b, c, d):

    n = len(d)

    a = np.asarray(a, dtype=float).copy()
    b = np.asarray(b, dtype=float).copy()
    c = np.asarray(c, dtype=float).copy()
    d = np.asarray(d, dtype=float).copy()

    w = np.zeros(n)
    g = np.zeros(n)
    x = np.zeros(n)

    if abs(b[0]) < 1e-15:
        raise ValueError("Pivote␣nulo␣en␣la␣primera␣ecuacion")

    w[0] = c[0] / b[0]
    g[0] = d[0] / b[0]

    for i in range(1, n):

        m = b[i] - a[i] * w[i-1]

        if abs(m) < 1e-15:
            raise ValueError("Pivote␣nulo␣durante␣Thomas")

        if i < n - 1:
            w[i] = c[i] / m

        g[i] = (d[i] - a[i] * g[i-1]) / m

    x[-1] = g[-1]

    for i in range(n-2, -1, -1):
        x[i] = g[i] - w[i] * x[i+1]
```

Aplicación de la función

```
import numpy as np

a = np.array([0, -1, -1, -1], dtype=float)
b = np.array([4,  4,  4,  4], dtype=float)
c = np.array([-1, -1, -1, 0], dtype=float)
d = np.array([3, 4, 5, 5], dtype=float)

x = thomas(a, b, c, d)

print("Solucion␣por␣Thomas:")
print(x)

# Matriz completa solamente para verificar
A = np.diag(b)
A += np.diag(c[:-1], 1)
A += np.diag(a[1:], -1)

x_np = np.linalg.solve(A, d)

print("\nSolucion␣de␣NumPy:")
print(x_np)

print("\nError:")
print(np.linalg.norm(x - x_np, np.inf))
```

Aplicación: ecuación de difusión

Consideremos la ecuación

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}.$$

Utilizando diferencias finitas:

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i-1}^{n+1} - 2u_i^{n+1} + u_{i+1}^{n+1}}{\Delta x^2}.$$

Con un esquema implícito:

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \alpha \frac{u_{i-1}^{n+1} - 2u_i^{n+1} + u_{i+1}^{n+1}}{\Delta x^2}.$$

Definiendo

$$r = \frac{\alpha \Delta t}{\Delta x^2},$$

obtenemos

$$-ru_{i-1}^{n+1} + (1+2r)u_i^{n+1} - ru_{i+1}^{n+1} = u_i^n.$$

Sistema tridiagonal en cada paso de tiempo
Para cada instante t^{n+1}, se obtiene

$$\begin{bmatrix} 1+2r & -r & 0 & \cdots & 0 \\ -r & 1+2r & -r & \ddots & \vdots \\ 0 & -r & 1+2r & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & -r \\ 0 & \cdots & 0 & -r & 1+2r \end{bmatrix} \begin{bmatrix} u_1^{n+1} \\ u_2^{n+1} \\ \vdots \\ u_{N-1}^{n+1} \\ u_N^{n+1} \end{bmatrix} =$$

Observación

El sistema es tridiagonal en cada paso temporal. Por tanto, el método de Thomas resulta especialmente apropiado.

Python: difusión 1D + Thomas

```
import numpy as np
import matplotlib.pyplot as plt

def thomas(a, b, c, d):

    n = len(d)
    w = np.zeros(n)
    g = np.zeros(n)
    x = np.zeros(n)

    w[0] = c[0] / b[0]
    g[0] = d[0] / b[0]

    for i in range(1, n):

        m = b[i] - a[i] * w[i-1]

        if i < n-1:
            w[i] = c[i] / m

        g[i] = (d[i] - a[i] * g[i-1]) / m

    x[-1] = g[-1]

    for i in range(n-2, -1, -1):
        x[i] = g[i] - w[i] * x[i+1]

    return x
```

Python: simulación de la difusión

```
L = 1.0
alpha = 1e-4

Nx = 20
Nt = 100
dt = 0.001
dx = L / Nx
r = alpha * dt / dx**2
x = np.linspace(0, L, Nx + 1)
u = np.ones(Nx + 1) * 100.0

# Condiciones de frontera
u[0] = 20.0
u[-1] = 20.0

# Coeficientes del sistema tridiagonal
N = Nx - 1

a = np.ones(N) * (-r)
b = np.ones(N) * (1 + 2*r)
c = np.ones(N) * (-r)

for n in range(Nt):
    d = u[1:-1].copy()
    # Aportes de las fronteras
    d[0]  += r * u[0]
    d[-1] += r * u[-1]
    u[1:-1] = thomas(a, b, c, d)

print("Temperatura␣final:")
print(u)
plt.plot(x, u, 'o-')
plt.xlabel("x")
plt.ylabel("Temperatura")
plt.grid()
plt.show()
```

Complejidad computacional

Para un sistema tridiagonal de dimensión n:

$$\boxed{O(n)}$$

operaciones aproximadamente.
Comparación:

Método	Complejidad aproximada	Estructura
Gauss	$O(n^3)$	General
Thomas	$O(n)$	Tridiagonal
Jacobi	$O(n^2)$ por iteración	General
Gauss-Seidel	$O(n^2)$ por iteración	General

Conclusión

Cuando el sistema es tridiagonal, utilizar Thomas permite aprovechar la estructura de la matriz y reducir considerablemente el costo computacional.

Consideraciones numéricas
Para aplicar correctamente el método de Thomas debemos prestar atención a los denominadores

$$b_i - a_i w_{i-1}.$$

Si alguno de ellos es cero o muy pequeño, pueden aparecer problemas numéricos.

- Verificar que los pivotes no sean cero.
- Evitar divisiones por números excesivamente pequeños.
- Analizar el condicionamiento del sistema.
- En problemas generales puede ser necesario utilizar pivoteo.

Importante

El método de Thomas es muy eficiente para sistemas tridiagonales bien comportados, especialmente aquellos provenientes de discretizaciones de ecuaciones diferenciales.

Thomas vs. eliminación de Gauss

Eliminación de Gauss

- Trabaja con matrices generales.
- Mayor almacenamiento.
- Mayor número de operaciones.
- Puede aprovecharse con pivoteo.

Thomas

- Diseñado para matrices tridiagonales.
- Almacenamiento $O(n)$.
- Operaciones $O(n)$.
- Implementación sencilla.

Si la matriz es tridiagonal, no tiene sentido ignorar su estructura.

Resumen del método

1. Identificar la estructura tridiagonal:
$$\mathbf{A} = \text{tridiag}(a_i, b_i, c_i).$$
2. Almacenar solamente las tres diagonales.
3. Realizar eliminación hacia adelante:
$$w_i = \frac{c_i}{b_i - a_i w_{i-1}},$$
$$g_i = \frac{d_i - a_i g_{i-1}}{b_i - a_i w_{i-1}}.$$
4. Calcular
$$x_n = g_n.$$
5. Realizar sustitución hacia atrás:
$$x_i = g_i - w_i x_{i+1}.$$
6. Verificar la solución mediante el residuo.

Aplicaciones del método de Thomas

El método de Thomas tiene numerosas aplicaciones en ingeniería:

- Transferencia de calor.
- Conducción térmica.
- Difusión de masa.
- Mecánica de fluidos.
- Ecuaciones de difusión.
- Ecuaciones de transporte.
- Problemas transitorios mediante diferencias finitas.
- Modelos unidimensionales de ingeniería.

En particular, aparece naturalmente cuando una ecuación diferencial parcial se discretiza en una dimensión espacial y conduce a una matriz tridiagonal.

Conclusiones

- El método de Thomas es una modificación especializada de la eliminación de Gauss.
- Está diseñado específicamente para sistemas tridiagonales.
- Solo requiere almacenar tres vectores.
- Su costo computacional es aproximadamente

$$O(n).$$

- Consta de dos etapas:
  1. eliminación hacia adelante;
  2. sustitución hacia atrás.
- Es especialmente útil en problemas de diferencias finitas asociados a ecuaciones diferenciales.

¿Preguntas?

Método de Thomas

"¿Para qué guardar una matriz completa
si solo necesitamos tres diagonales?"

Método de Relajación Sucesiva ((SOR)

El método SOR puede interpretarse como una modificación de Gauss–Seidel:

$$\boxed{\text{SOR} = \text{Gauss–Seidel} + \text{relajación}}$$

La relajación permite acelerar la convergencia en muchos problemas.

Sistema de ecuaciones lineales
Consideremos el sistema

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2, \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n. \end{aligned}$$

Despejando x_i:

$$x_i = \frac{1}{a_{ii}} \left[b_i - \sum_{j=1}^{i-1} a_{ij}x_j - \sum_{j=i+1}^{n} a_{ij}x_j \right].$$

En Gauss–Seidel se utilizan inmediatamente los valores actualizados.

Método de Gauss–Seidel
La fórmula de Gauss–Seidel es

$$\boxed{x_i^{(k+1)} = \frac{1}{a_{ii}} \left[b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k)} \right]}$$

Observe que:

$$\begin{cases} x_j^{(k+1)}, & j < i, \\ x_j^{(k)}, & j > i. \end{cases}$$

Es decir, los valores nuevos se utilizan inmediatamente.

Limitación

Aunque Gauss–Seidel puede converger mucho más rápido que Jacobi, en determinados problemas todavía puede requerir muchas iteraciones.

Idea de relajación

Supongamos que Gauss–Seidel produce

$$x_i^{GS}.$$

En SOR no necesariamente aceptamos directamente ese valor. Definimos

$$\boxed{x_i^{(k+1)} = (1-\omega)x_i^{(k)} + \omega x_i^{GS}}$$

donde

$$\boxed{\omega = \text{parámetro de relajación.}}$$

Por tanto:

$$\omega = 1 \implies \text{Gauss–Seidel.}$$

$$0 < \omega < 1 \implies \text{subrelajación.}$$

$$1 < \omega < 2 \implies \text{sobrerrelajación.}$$

Interpretación del parámetro ω
La actualización SOR puede escribirse como

$$x_i^{(k+1)} = x_i^{(k)} + \omega\left(x_i^{GS} - x_i^{(k)}\right).$$

Por tanto, ω controla cuánto nos desplazamos desde el valor anterior hacia el valor calculado por Gauss–Seidel.

$0 < \omega < 1$

Subrelajación

- Actualización conservadora.
- Puede estabilizar procesos difíciles.
- Generalmente más lenta.

$\omega = 1$

Gauss–Seidel

- Sin relajación.
- Caso particular de SOR.

$1 < \omega < 2$

Sobrerrelajación

- Busca acelerar la convergencia.
- Requiere elegir adecuadamente ω.

Derivación de la fórmula SOR

Partimos de Gauss–Seidel:

$$x_i^{GS} = \frac{1}{a_{ii}} \left[b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right].$$

Aplicamos relajación:

$$x_i^{(k+1)} = (1-\omega) x_i^{(k)} + \omega x_i^{GS}.$$

Por tanto,

$$\boxed{x_i^{(k+1)} = (1-\omega) x_i^{(k)} + \frac{\omega}{a_{ii}} \left[b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right]}$$

Esta es la fórmula fundamental del método SOR.

Forma matricial

Descomponemos la matriz:

$$\mathbf{A} = \mathbf{D} + \mathbf{L} + \mathbf{U},$$

donde:

$$\mathbf{D} = \text{diagonal},$$
$$\mathbf{L} = \text{parte triangular inferior},$$
$$\mathbf{U} = \text{parte triangular superior}.$$

El método SOR puede expresarse como

$$\boxed{(\mathbf{D} + \omega \mathbf{L}) \mathbf{x}^{(k+1)} = \omega \mathbf{b} - [\omega \mathbf{U} + (\omega - 1)\mathbf{D}] \mathbf{x}^{(k)}}$$

o equivalentemente,

$$\boxed{\mathbf{x}^{(k+1)} = \mathbf{T}_\omega \mathbf{x}^{(k)} + \mathbf{c}_\omega}$$

Matriz de iteración SOR

La matriz de iteración es

$$\boxed{\mathbf{T}_\omega = -(\mathbf{D} + \omega\mathbf{L})^{-1}\left[\omega\mathbf{U} + (\omega - 1)\mathbf{D}\right]}$$

y

$$\mathbf{c}_\omega = \omega(\mathbf{D} + \omega\mathbf{L})^{-1}\mathbf{b}.$$

La convergencia está relacionada con el radio espectral:

$$\boxed{\rho(\mathbf{T}_\omega) < 1}$$

donde

$$\rho(\mathbf{T}) = \max_i |\lambda_i|.$$

Interpretación

Cuanto menor sea el radio espectral, más rápida suele ser la convergencia asintótica.

Algoritmo SOR

Datos de entrada:

$\mathbf{A}$, $\mathbf{b}$, $\mathbf{x}^{(0)}$, ω, ε, $N_{\text{máx}}$.

Procedimiento:

1. Seleccionar una aproximación inicial $\mathbf{x}^{(0)}$.
2. Para $k = 0, 1, 2, \ldots$:
3. Actualizar $x_1, x_2, \ldots, x_n$.
4. Calcular el error.
5. Verificar el criterio de convergencia.
6. Detener el proceso si se alcanza la tolerancia.

Pseudocódigo del método SOR

```
Entrada:
    A, b
    x0, omega
    tolerancia, iter_max

x = x0

Para k = 1,...,iter_max:
    x_old = x

    Para i = 1,...,n:
        suma1 = sum(A[i,j]*x[j], j = 1,...,i-1)
        suma2 = sum(A[i,j]*x_old[j], j = i+1,...,n)
        x_GS = (b[i] - suma1 - suma2)/A[i,i]
        x[i] = (1 - omega)*x_old[i] + omega*x_GS

    error = norma(x - x_old)/norma(x)

    Si error < tolerancia:
        detener

Salida:
    x, número de iteraciones, error
```

Criterios de convergencia

Existen diferentes criterios para detener el proceso.

Criterio basado en el cambio relativo:

$$\varepsilon^{(k)} = \frac{\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|_\infty}{\|\mathbf{x}^{(k+1)}\|_\infty}$$

Se detiene cuando

$$\varepsilon^{(k)} < \varepsilon_{\text{tol}}$$

Otra alternativa es utilizar el residuo:

$$\mathbf{r}^{(k)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}$$

y verificar

$$\|\mathbf{r}^{(k)}\|_\infty < \varepsilon_{\text{tol}}.$$

Ejemplo 1: sistema 3×3

Resolver mediante SOR el sistema

$$\begin{aligned} 10x_1 - x_2 &= 9, \\ -x_1 + 10x_2 - x_3 &= 8, \\ -x_2 + 10x_3 &= 9. \end{aligned}$$

En forma matricial:

$$\begin{bmatrix} 10 & -1 & 0 \\ -1 & 10 & -1 \\ 0 & -1 & 10 \end{bmatrix} \begin{Bmatrix} x_1 \\ x_2 \\ x_3 \end{Bmatrix} = \begin{Bmatrix} 9 \\ 8 \\ 9 \end{Bmatrix}.$$

Tomamos

$$\mathbf{x}^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \quad \text{y} \quad \omega = 1{,}5$$

Usaremos

$$\varepsilon_{\text{tol}} = 10^{-6}.$$

Ecuaciones iterativas

De la primera ecuación:

$$x_1^{GS} = \frac{9 + x_2}{10}.$$

De la segunda:

$$x_2^{GS} = \frac{8 + x_1 + x_3}{10}.$$

De la tercera:

$$x_3^{GS} = \frac{9 + x_2}{10}.$$

Las ecuaciones SOR son:

$$x_1^{(k+1)} = (1-\omega)x_1^{(k)} + \omega \frac{9 + x_2^{(k)}}{10}$$

$$x_2^{(k+1)} = (1-\omega)x_2^{(k)} + \omega \frac{8 + x_1^{(k+1)} + x_3^{(k)}}{10}$$

$$x_3^{(k+1)} = (1-\omega)x_3^{(k)} + \omega \frac{9 + x_2^{(k+1)}}{10}$$

Ejemplo 1: primera y segunda iteración

primera iteración

Tenemos

$$x_1^{(0)} = x_2^{(0)} = x_3^{(0)} = 0 \quad \text{y} \quad \omega = 1{,}5$$

Para x_1:

$$x_1^{(1)} = (1 - 1{,}5)(0) + 1{,}5\left(\frac{9 + 0}{10}\right)$$

$$x_1^{(1)} = 1{,}35$$

Para x_2:

$$x_2^{(1)} = (1 - 1{,}5)(0) + 1{,}5\left(\frac{8 + 1{,}35 + 0}{10}\right)$$

$$x_2^{(1)} = 1{,}4025$$

Para x_3:

$$x_3^{(1)} = (1 - 1{,}5)(0) + 1{,}5\left(\frac{9 + 1{,}4025}{10}\right)$$

$$x_3^{(1)} = 1{,}560375$$

Por tanto,

$$\mathbf{x}^{(1)} = \begin{bmatrix} 1{,}35 \\ 1{,}4025 \\ 1{,}560375 \end{bmatrix}$$

segunda iteración

Para x_1:

$$x_1^{(2)} = -0{,}5(1{,}35) + 1{,}5\left(\frac{9 + 1{,}4025}{10}\right)$$

$$x_1^{(2)} = 0{,}831875$$

Para x_2:

$$x_2^{(2)} = -0{,}5(1{,}4025) + 1{,}5\left(\frac{8 + 0{,}831875 + 1{,}560375}{10}\right)$$

$$x_2^{(2)} \approx 1{,}17431$$

Para x_3:

$$x_3^{(2)} = -0{,}5(1{,}560375) + 1{,}5\left(\frac{9 + 1{,}17431}{10}\right)$$

$$x_3^{(2)} \approx 0{,}74459$$

Por tanto,

$$\mathbf{x}^{(2)} \approx \begin{bmatrix} 0{,}83188 \\ 1{,}17431 \\ 0{,}74459 \end{bmatrix}.$$

Ejemplo 1: solución

Continuando el proceso iterativo, obtenemos:

k	x_1	x_2	x_3	ε
0	0	0	0	–
1	1,3500	1,4025	1,5604	–
2	0,8319	1,1743	0,7446	–
3	1,0189	0,9270	1,1110	–
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Después de suficientes iteraciones:

$$\left.\begin{aligned} x_1 &\approx 1{,}000000 \\ x_2 &\approx 0{,}999999 \\ x_3 &\approx 1{,}000000 \end{aligned}\right\} \quad \Rightarrow \quad \mathbf{x} \approx \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

Observación

El método SOR no busca la solución exacta en una sola operación. Construye progresivamente una aproximación hasta satisfacer la tolerancia especificada.

Python: implementación de SOR

```python
import numpy as np

def sor(A, b, x0, omega, tol=1e-8, max_iter=10000):

    A = np.asarray(A, dtype=float)
    b = np.asarray(b, dtype=float)
    x = np.asarray(x0, dtype=float).copy()

    n = len(b)

    for k in range(1, max_iter + 1):

        x_old = x.copy()

        for i in range(n):

            suma1 = np.dot(A[i, :i], x[:i])
            suma2 = np.dot(A[i, i+1:], x_old[i+1:])

            x_gs = (b[i] - suma1 - suma2) / A[i, i]

            x[i] = ((1 - omega) * x_old[i]
                    + omega * x_gs)

        error = np.linalg.norm(x - x_old, np.inf) \
                / np.linalg.norm(x, np.inf)

        if error < tol:
            return x, k, error

    return x, max_iter, error
```

```python
A = np.array([
    [10, -1,  0],
    [-1, 10, -1],
    [0,  -1, 10]
], dtype=float)

b = np.array([9, 8, 9], dtype=float)

x0 = np.zeros(3)

omega = 1.5

x, iterations, error = sor(
    A, b, x0, omega,
    tol=1e-8
)

print("Solucion:")
print(x)

print("Iteraciones:", iterations)
print("Error:", error)

print("Residuo:")
print(b - A @ x)
```

Resultado esperado:

$$\mathbf{x} \approx \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

Ejemplo 2: ecuación de Laplace

Consideremos una placa rectangular en estado estacionario.
La temperatura satisface:

$$\boxed{\nabla^2 T = 0}$$

es decir,

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0.$$

Supongamos las condiciones:

$$T = 100^\circ C \qquad \text{en el borde inferior,}$$

$$T = 0^\circ C \qquad \text{en los otros tres bordes.}$$

El objetivo es determinar la temperatura en los nodos interiores.

Aplicación en Ingeniería Mecánica

Este procedimiento aparece en problemas de conducción térmica, transferencia de calor y análisis de campos estacionarios.

Discretización mediante diferencias finitas
Para una malla uniforme:

$$\Delta x = \Delta y = h.$$

La ecuación de Laplace se aproxima mediante:

$$\frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{h^2} + \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{h^2} = 0.$$

Multiplicando por h^2:

$$T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1} - 4T_{i,j} = 0.$$

Por tanto:

$$\boxed{T_{i,j} = \frac{1}{4}\left(T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1}\right)}$$

Esta es la ecuación de actualización de Gauss–Seidel.

SOR aplicado a la ecuación de Laplace

Definimos primero el valor obtenido mediante Gauss–Seidel:

$$T_{i,j}^{GS} = \frac{1}{4}\left(T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1}\right)$$

Aplicando SOR:

$$T_{i,j}^{(k+1)} = (1-\omega)T_{i,j}^{(k)} + \omega T_{i,j}^{GS}$$

$$T_{i,j}^{(k+1)} = (1-\omega)T_{i,j}^{(k)} + \frac{\omega}{4}\left(T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1}\right)$$

Los valores ya actualizados se utilizan inmediatamente.

primera iteración

Para simplificar, consideremos cuatro nodos interiores:

$$\begin{array}{cc} T_{1,2} & T_{2,2} \\ T_{1,1} & T_{2,1} \end{array}$$

con

$$T = 100^\circ C$$

en el borde inferior y

$$T = 0^\circ C$$

en los demás bordes.

Tomamos:

$$T_{i,j}^{(0)} = 0, \qquad \omega = 1{,}5.$$

Para el primer nodo interior:

$$T_{1,1}^{GS} = \frac{100+0+0+0}{4} = 25^\circ C.$$

Entonces:

$$T_{1,1}^{(1)} = (1-1{,}5)(0) + 1{,}5(25)$$

$$T_{1,1}^{(1)} = 37{,}5^\circ C$$

El proceso continúa nodo por nodo.

Para cada iteración:

$$(i,j) = (1,1), (2,1), \ldots$$

se calcula:

$$T_{i,j}^{GS} = \frac{T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1}}{4}.$$

Luego:

$$T_{i,j}^{new} = (1-\omega)T_{i,j}^{old} + \omega T_{i,j}^{GS}.$$

Finalmente:

$$\varepsilon = \max_{i,j} \left| T_{i,j}^{new} - T_{i,j}^{old} \right|$$

El proceso continúa hasta que

$$\varepsilon < \varepsilon_{\text{tol}}.$$

Python: SOR para la ecuación de Laplace

```python
import numpy as np

def sor_laplace(nx, ny, omega,
                tol=1e-6, max_iter=10000):

    T = np.zeros((ny, nx))

    # Borde inferior
    T[0, :] = 100.0

    for iteration in range(1, max_iter + 1):

        T_old = T.copy()

        for j in range(1, ny - 1):

            for i in range(1, nx - 1):

                T_gs = 0.25 * (
                    T[j, i+1] +
                    T[j, i-1] +
                    T[j+1, i] +
                    T[j-1, i]
                )

                T[j, i] = (
                    (1 - omega) * T_old[j, i]
                    + omega * T_gs
                )

        error = np.max(np.abs(T - T_old))

        if error < tol:
            return T, iteration, error

    return T, max_iter, error
```

Python: ejecución del Ejemplo 2

```python
nx = 21
ny = 21

omega = 1.7

T, iterations, error = sor_laplace(
    nx, ny,
    omega,
    tol=1e-6
)

print("Iteraciones:", iterations)
print("Error:", error)

print("Temperatura central:")
print(T[ny//2, nx//2])
```

El programa calcula:

$$\boxed{T(x, y)}$$

en todos los nodos de la malla.

Visualización de la solución

```
import matplotlib.pyplot as plt

x = np.linspace(0, 1, nx)
y = np.linspace(0, 1, ny)

X, Y = np.meshgrid(x, y)

plt.figure(figsize=(7,5))

contour = plt.contourf(
    X, Y, T, levels=20
)

plt.colorbar(contour, label="Temperatura [$^\
      circ$C]")

plt.xlabel("x")
plt.ylabel("y")
plt.title("Solucion de Laplace mediante SOR")

plt.tight_layout()
plt.show()
```

La solución permite observar la distribución de temperatura en toda la placa.

Interpretación física del resultado

La solución numérica representa el campo

$$T = T(x, y)$$

En régimen estacionario:

$$\nabla^2 T = 0$$

El calor fluye desde las regiones de mayor temperatura hacia las regiones de menor temperatura.
El gradiente térmico es:

$$\boxed{\mathbf{q} = -k\nabla T}$$

donde k es la conductividad térmica.
Por tanto, SOR no solamente proporciona una solución algebraica:

SOR $\longrightarrow$ temperaturas nodales $\longrightarrow$ campo térmic

¿Cómo seleccionar ω?

La elección de ω es fundamental.

$$\boxed{0 < \omega < 2}$$

es el intervalo usual de interés para SOR.

$$\omega = 1$$

corresponde a Gauss–Seidel. Para muchos problemas:

$$1 < \omega < 2$$

puede acelerar considerablemente la convergencia. Sin embargo:

$$\omega \text{ demasiado grande}$$

puede producir oscilaciones o incluso impedir la convergencia.

Idea importante

No existe un único valor de ω que sea óptimo para todos los problemas.

Comparación de diferentes valores de ω
Para un mismo problema podemos ejecutar:

$$\omega = 0{,}8, \qquad \omega = 1{,}0, \qquad \omega = 1{,}2, \qquad \omega = 1{,}5, \qquad \omega = 1{,}7, \qquad \omega = 1{,}9.$$

y registrar:

ω	Iteraciones	Convergencia
0,8	mayor	✓
1,0	–	✓
1,2	menor	✓
1,5	menor	✓
1,7	–	✓
1,9	posiblemente mayor	✓

El valor óptimo debe determinarse para el problema concreto.

Python: búsqueda del mejor ω

```
omegas = np.arange(0.5, 1.95, 0.05)

resultados = []

for omega in omegas:

    T, it, error = sor_laplace(
        nx=21,
        ny=21,
        omega=omega,
        tol=1e-6
    )

    resultados.append((omega, it))

for omega, it in resultados:
    print(f"omega = {omega:.2f}, "
          f"iteraciones = {it}")
```

El menor número de iteraciones proporciona una estimación del valor de ω más eficiente para esta malla y esta tolerancia.

Gráfica de convergencia

Podemos almacenar el error en cada iteración:

$$\varepsilon^{(1)}, \varepsilon^{(2)}, \ldots, \varepsilon^{(k)}.$$

```
plt.figure(figsize=(7,5))

plt.semilogy(
    errores,
    marker='o',
    markersize=3
)

plt.xlabel("Iteracion")
plt.ylabel("Error")
plt.title("Convergencia del metodo SOR")
plt.grid(True)

plt.tight_layout()
plt.show()
```

La escala logarítmica permite observar mejor la reducción del error.

SOR en elementos finitos

En elementos finitos obtenemos:

$$\boxed{\mathbf{KQ} = \mathbf{F}}$$

donde:

$$\mathbf{K} = \text{matriz global de rigidez},$$

$$\mathbf{Q} = \text{vector de desplazamientos},$$

$$\mathbf{F} = \text{vector global de cargas}.$$

SOR puede utilizarse para resolver iterativamente:

$$\mathbf{KQ} = \mathbf{F}.$$

En cada iteración se actualizan los desplazamientos nodales:

$$Q_1, Q_2, \ldots, Q_n.$$

Aplicación

Para sistemas grandes y dispersos, los métodos iterativos pueden ser atractivos porque evitan almacenar o factorizar completamente la matriz global.

Interpretación mecánica
En un problema estructural:

$$\mathbf{KQ} = \mathbf{F}.$$

SOR genera:

$$\mathbf{Q}^{(0)} \to \mathbf{Q}^{(1)} \to \mathbf{Q}^{(2)} \to \cdots \to \mathbf{Q}.$$

Cada componente representa un grado de libertad:

$$Q_i = \begin{cases} u_i, & \text{desplazamiento en } x, \\ v_i, & \text{desplazamiento en } y, \\ w_i, & \text{desplazamiento en } z, \\ \theta_i, & \text{rotación}. \end{cases}$$

La convergencia implica que:

$$\boxed{\mathbf{KQ}^{(k)} \to \mathbf{F}}$$

Ventajas y limitaciones de SOR

Ventajas

- Fácil de implementar.
- Bajo costo por iteración.
- Puede aprovechar matrices dispersas.
- Puede ser mucho más rápido que Gauss–Seidel.
- Adecuado para problemas provenientes de diferencias finitas.

Limitaciones

- Requiere una elección adecuada de ω.
- Puede converger lentamente.
- No converge para cualquier matriz.
- Puede ser sensible a la numeración y estructura del problema.
- Para sistemas grandes existen métodos más avanzados.

Resumen del método SOR

El método SOR se obtiene modificando Gauss–Seidel:

$$x_i^{(k+1)} = (1-\omega)x_i^{(k)} + \frac{\omega}{a_{ii}}\left[b_i - \sum_{j<i} a_{ij}x_j^{(k+1)} - \sum_{j>i} a_{ij}x_j^{(k)}\right]$$

Casos principales:

$0 < \omega < 1$	:	subrelajación
$\omega = 1$	:	Gauss–Seidel
$1 < \omega < 2$	:	sobrerrelajación

La convergencia se controla mediante:

$$\varepsilon = \frac{\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|}{\|\mathbf{x}^{(k+1)}\|}$$

$$\boxed{\text{SOR} = \text{Gauss–Seidel} + \text{relajación}}$$

El objetivo no es solamente obtener una solución,

$$\mathbf{x},$$

sino obtenerla de manera

$$\boxed{\text{eficiente} + \text{estable} + \text{con una precisión controlada.}}$$

En problemas de Ingeniería Mecánica:

$$\boxed{\mathbf{KQ} = \mathbf{F}}$$

constituye una de las aplicaciones fundamentales.

Ejercicio

Para la matriz

$$\mathbf{A} = \begin{bmatrix} 4 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 4 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 15 \\ 10 \\ 10 \end{bmatrix},$$

resolver mediante SOR utilizando:

$$\mathbf{x}^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

Comparar:

$$\omega = 0{,}8, \qquad \omega = 1{,}0, \qquad \omega = 1{,}2, \qquad \omega = 1{,}5.$$

Para cada caso determine:

1. La solución.
2. El número de iteraciones.
3. El error final.
4. El residuo.

Entrega

Implementar el algoritmo desde cero en Python, sin utilizar una función predeterminada de SOR.

¿Preguntas?

SOR
Gauss-Seidel + Relajación

¿Por qué utilizar un método iterativo?

Gradiente Conjugado (CG)

Supongamos que el modelo de Elementos Finitos tiene $\mathbf{N} = 10^6$ **grados de libertad**. Una matriz global puede tener dimensiones

$$\mathbf{K} \in \mathbb{R}^{N \times N}.$$

Sin embargo, debido a la conectividad de los elementos,

$$\mathbf{K}$$

es normalmente una matriz dispersa.

Idea fundamental

No necesitamos almacenar ni operar con todos los elementos de $\mathbf{K}$. Podemos aprovechar su estructura dispersa mediante métodos iterativos.

El método del Gradiente Conjugado (CG) es especialmente apropiado cuando

$$\boxed{\mathbf{K} = \mathbf{K}^T, \qquad \mathbf{K} > 0}$$

es decir, cuando la **matriz es simétrica**

Sistema lineal y función cuadrática
Consideremos el sistema

$$\boxed{\mathbf{Ax} = \mathbf{b}}$$

con

$$\mathbf{A} = \mathbf{A}^T$$

y **definida positiva.**
Definimos la función cuadrática

$$\boxed{\phi(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T\mathbf{Ax} - \mathbf{b}^T\mathbf{x}}$$

Su gradiente es

$$\nabla\phi(\mathbf{x}) = \mathbf{Ax} - \mathbf{b}.$$

Por tanto,

$$\nabla\phi(\mathbf{x}) = 0 \Rightarrow \mathbf{Ax} = \mathbf{b}.$$

Interpretación

Resolver el sistema lineal equivale a encontrar el mínimo de una función cuadrática.

Interpretación energética

En Elementos Finitos, la **energía potencial total** puede escribirse como

$$\Pi(\mathbf{Q}) = \frac{1}{2}\mathbf{Q}^T\mathbf{K}\mathbf{Q} - \mathbf{Q}^T\mathbf{F}.$$

La **condición de equilibrio es**

$$\frac{\partial \Pi}{\partial \mathbf{Q}} = 0.$$

Por lo tanto,

$$\mathbf{K}\mathbf{Q} - \mathbf{F} = 0,$$

y obtenemos

$$\boxed{\mathbf{K}\mathbf{Q} = \mathbf{F}}.$$

El **Gradiente Conjugado (CG)** puede interpretarse como un **procedimiento iterativo para minimizar**

$$\boxed{\Pi(\mathbf{Q}) = \frac{1}{2}\mathbf{Q}^T\mathbf{K}\mathbf{Q} - \mathbf{Q}^T\mathbf{F}}$$

sin calcular explícitamente la inversa de $\mathbf{K}$.

Residuo y dirección de descenso
Para una aproximación $\mathbf{x}_k$, definimos el residuo

$$\boxed{\mathbf{r}_k = \mathbf{b} - \mathbf{A}\mathbf{x}_k}$$

El residuo representa el error de equilibrio del sistema.
Como

$$\nabla\phi(\mathbf{x}_k) = \mathbf{A}\mathbf{x}_k - \mathbf{b},$$

tenemos

$$\boxed{\mathbf{r}_k = -\nabla\phi(\mathbf{x}_k)}$$

Por tanto, $\mathbf{r}_k$ es una dirección de descenso de la función ϕ.
El método del descenso más pronunciado utilizaría directamente

$$\mathbf{p}_k = \mathbf{r}_k.$$

El **Gradiente Conjugado (CG)** mejora esta estrategia utilizando direcciones cuidadosamente construidas.

Conjugación de las direcciones

Dos vectores $\mathbf{p}_i$ y $\mathbf{p}_j$ son **A**-conjugados si

$$\boxed{\mathbf{p}_i^T \mathbf{A}\mathbf{p}_j = 0, \qquad i \neq j.}$$

En el **método CG** se construye una secuencia de direcciones

$$\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2, \ldots$$

que cumplen

$$\mathbf{p}_i^T \mathbf{A}\mathbf{p}_j = 0.$$

Esta propiedad es fundamental porque permite avanzar hacia la solución sin destruir el progreso realizado en las direcciones anteriores.

Ventaja

En aritmética exacta, para una **matriz** $n \times n$ **definida positiva, CG converge en como máximo** n **iteraciones.**

En la práctica, normalmente se obtiene una solución suficientemente precisa mucho antes.

Algoritmo del Gradiente Conjugado

Partimos de una aproximación inicial

$$\mathbf{x}_0$$

Calculamos el residuo inicial

$$\mathbf{r}_0 = \mathbf{b} - \mathbf{A}\mathbf{x}_0.$$

La primera dirección es

$$\mathbf{p}_0 = \mathbf{r}_0$$

Para $k = 0, 1, 2, \ldots$:

$$\alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T \mathbf{A}\mathbf{p}_k}$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$$

$$\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k \mathbf{A}\mathbf{p}_k$$

y

$$\beta_k = \frac{\mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{r}_k^T \mathbf{r}_k}$$

Finalmente,

$$\mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k$$

Algoritmo completo

Gradiente Conjugado

Entrada:

$$\mathbf{A}, \quad \mathbf{b}, \quad \mathbf{x}_0, \quad \varepsilon$$

1. Calcular
$$\mathbf{r}_0 = \mathbf{b} - \mathbf{A}\mathbf{x}_0.$$
2. Inicializar
$$\mathbf{p}_0 = \mathbf{r}_0.$$
3. Para $k = 0, 1, \ldots$:
$$\alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T \mathbf{A}\mathbf{p}_k}.$$
$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k.$$
$$\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k \mathbf{A}\mathbf{p}_k.$$
4. Si
$$\|\mathbf{r}_{k+1}\| < \varepsilon,$$
detenerse.
5. Calcular β_k y actualizar $\mathbf{p}_{k+1}$.

Pseudocódigo del método CG

$$\mathbf{r} = \mathbf{b} - \mathbf{A}\mathbf{x}$$
$$\mathbf{p} = \mathbf{r}$$

Mientras

$$\|\mathbf{r}\| > \varepsilon :$$

$$\mathbf{q} = \mathbf{A}\mathbf{p}$$
$$\alpha = \frac{\mathbf{r}^T \mathbf{r}}{\mathbf{p}^T \mathbf{q}}$$
$$\mathbf{x} \leftarrow \mathbf{x} + \alpha \mathbf{p}$$
$$\mathbf{r}_{\text{nuevo}} \leftarrow \mathbf{r} - \alpha \mathbf{q}$$
$$\beta = \frac{\mathbf{r}_{\text{nuevo}}^T \mathbf{r}_{\text{nuevo}}}{\mathbf{r}^T \mathbf{r}}$$
$$\mathbf{p} \leftarrow \mathbf{r}_{\text{nuevo}} + \beta \mathbf{p}$$

Ejemplo 1: sistema 2×2

Resolver mediante Gradiente Conjugado:

$$\begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix} \begin{Bmatrix} x_1 \\ x_2 \end{Bmatrix} = \begin{Bmatrix} 1 \\ 2 \end{Bmatrix}.$$

Por tanto,

$$\mathbf{A} = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

La matriz es simétrica:

$$\mathbf{A}^T = \mathbf{A}.$$

Además, para demostrar que es **definida positiva**, debemos verificar:

$$\mathbf{z}^T \mathbf{A}\mathbf{z} > 0$$

Calculamos:

$$\mathbf{A}\mathbf{z} = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 4x + y \\ x + 3y \end{bmatrix}$$

$$\mathbf{z}^T \mathbf{A}\mathbf{z} = \begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} 4x + y \\ x + 3y \end{bmatrix}$$

Continuamos con el producto:

$$\begin{aligned} \mathbf{z}^T \mathbf{A}\mathbf{z} &= x(4x + y) + y(x + 3y) \\ &= 4x^2 + xy + xy + 3y^2 \\ \mathbf{z}^T \mathbf{A}\mathbf{z} &= 4x^2 + 2xy + 3y^2 \end{aligned}$$

Ahora debemos demostrar que esta expresión es estrictamente positiva para todo $\mathbf{z} \neq \mathbf{0}$. Completamos cuadrados:

$$\begin{aligned} 4x^2 + 2xy + 3y^2 &= 4\left(x^2 + \frac{1}{2}xy\right) + 3y^2 \\ &= 4\left(x + \frac{y}{4}\right)^2 + \frac{11}{4}y^2. \end{aligned}$$

Así,

$$\boxed{\mathbf{z}^T \mathbf{A}\mathbf{z} = 4\left(x + \frac{y}{4}\right)^2 + \frac{11}{4}y^2}$$

Ambos términos son ≥ 0. Tenemos:

$$4\left(x + \frac{y}{4}\right)^2 \geq 0 \quad \text{y} \quad \frac{11}{4}y^2 \geq 0,$$

la suma nunca puede ser negativa. Además, para que la suma sea igual a cero, ambos términos deberían ser simultáneamente cero:

$$y = 0 \quad \text{y} \quad x + \frac{y}{4} = 0.$$

para todo $\mathbf{z} \neq 0$. Por tanto,

$$\boxed{\textbf{A es simétrica definida positiva.}}$$

Ejemplo 1: sistema 2×2

Otra forma de verificar la definición positiva es mediante los **autovalores**.

Calculamos:

$$\det(\mathbf{A} - \lambda \mathbf{I}) = 0.$$

Entonces:

$$\begin{vmatrix} 4-\lambda & 1 \\ 1 & 3-\lambda \end{vmatrix} = 0(4-\lambda)(3-\lambda) - 1 = 0.$$

Los autovalores son:

$$\lambda^2 - 7\lambda + 11 = 0. \Rightarrow \boxed{\lambda_{1,2} = \frac{7 \pm \sqrt{5}}{2}}$$

Como:

$$\lambda_1 \approx 4{,}6181 > 0, \qquad \lambda_2 \approx 2{,}382 > 0, .$$

todos los **autovalores son positivos.**
Por lo tanto,

A es simétrica definida positiva.

Iteración inicial: El residuo inicial es

$$\mathbf{r}_0 = \mathbf{b} - \mathbf{A}\mathbf{x}_0 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Por tanto,

$$\mathbf{p}_0 = \mathbf{r}_0 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Calculamos

$$\mathbf{A}\mathbf{p}_0 = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 6 \\ 7 \end{bmatrix}.$$

Entonces,

$$\alpha_0 = \frac{\mathbf{r}_0^T \mathbf{r}_0}{\mathbf{p}_0^T \mathbf{A}\mathbf{p}_0} = \frac{1^2 + 2^2}{1(6) + 2(7)} = \frac{5}{20} = \boxed{0{,}25}.$$

Primera actualización: Actualizamos la solución:

$$\mathbf{x}_1 = \mathbf{x}_0 + \alpha_0 \mathbf{p}_0.$$

Por tanto,

$$\mathbf{x}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + 0{,}25 \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 0{,}25 \\ 0{,}50 \end{bmatrix}$$

El nuevo residuo es

$$\mathbf{r}_1 = \mathbf{r}_0 - \alpha_0 \mathbf{A}\mathbf{p}_0 = \begin{bmatrix} 1 \\ 2 \end{bmatrix} - 0{,}25 \begin{bmatrix} 6 \\ 7 \end{bmatrix} = \begin{bmatrix} -0{,}5 \\ 0{,}25 \end{bmatrix}$$

Ejemplo 1:

Segunda dirección: Calculamos

$$\beta_0 = \frac{\mathbf{r}_1^T \mathbf{r}_1}{\mathbf{r}_0^T \mathbf{r}_0}.$$

Tenemos

$$\mathbf{r}_1^T \mathbf{r}_1 = (-0{,}5)^2 + (0{,}25)^2 = 0{,}3125.$$

Por tanto,

$$\beta_0 = \frac{0{,}3125}{5} = \boxed{0{,}0625}.$$

La nueva dirección es

$$\mathbf{p}_1 = \mathbf{r}_1 + \beta_0 \mathbf{p}_0 = \begin{bmatrix} -0{,}5 \\ 0{,}25 \end{bmatrix} + 0{,}0625 \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} -0{,}4375 \\ 0{,}375 \end{bmatrix}$$

Segunda iteración: Calculamos

$$\mathbf{A}\mathbf{p}_1 = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} -0{,}4375 \\ 0{,}375 \end{bmatrix} = \begin{bmatrix} -1{,}375 \\ 0{,}6875 \end{bmatrix}.$$

Entonces,

$$\alpha_1 = \frac{\mathbf{r}_1^T \mathbf{r}_1}{\mathbf{p}_1^T \mathbf{A}\mathbf{p}_1} = 0{,}363636$$

Por tanto,

$$\mathbf{x}_2 = \mathbf{x}_1 + \alpha_1 \mathbf{p}_1 = \begin{bmatrix} 0{,}090909 \\ 0{,}636364 \end{bmatrix}$$

Solución exacta: La solución exacta puede verificarse directamente:

$$\begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 1/11 \\ 7/11 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Resultado

Por tanto,

$$\mathbf{x} = \begin{bmatrix} 1/11 \\ 7/11 \end{bmatrix} = \begin{bmatrix} 0{,}090909 \\ 0{,}636364 \end{bmatrix}$$

El método CG converge en dos iteraciones porque el sistema tiene solamente dos grados de libertad.

Ejemplo 1: implementación en Python

```
import numpy as np

A = np.array([[4., 1.],
              [1., 3.]])
b = np.array([1., 2.])
x = np.zeros(2)

r = b - A @ x
p = r.copy()

tol = 1e-10  # tolerancia

for k in range(10):
    Ap = A @ p
    alpha = (r @ r) / (p @ Ap)
    x = x + alpha * p
    r_new = r - alpha * Ap
    print(k+1, x, np.linalg.norm(r_new))

    if np.linalg.norm(r_new) < tol:
        break

    beta = (r_new @ r_new) / (r @ r)
    p = r_new + beta * p
    r = r_new

print("Solucion:", x)
```

Resultados computacionales: Una posible tabla de convergencia es

Iteración	$\mathbf{x}_k$	$\|\mathbf{r}_k\|$
0	$(0, 0)$	2,2361
1	$(0{,}25, 0{,}50)$	0,5590
2	$(0{,}090909, 0{,}636364)$	≈ 0

La norma del residuo disminuye hasta alcanzar el criterio

$$\boxed{\|\mathbf{r}_k\| < 10^{-10}}.$$

Observación

El método no necesita calcular $\mathbf{A}^{-1}$.

Ejemplo 2: sistema proveniente de Elementos Finitos

Consideremos el sistema reducido de una estructura:

$$\boxed{\mathbf{KQ} = \mathbf{F}}$$

con

$$\mathbf{K} = \begin{bmatrix} 10 & -2 & 0 & 0 \\ -2 & 10 & -2 & 0 \\ 0 & -2 & 10 & -2 \\ 0 & 0 & -2 & 8 \end{bmatrix}$$

y

$$\mathbf{F} = \begin{bmatrix} 8 \\ 6 \\ 4 \\ 10 \end{bmatrix}.$$

Interpretación:

- $\mathbf{K}$: matriz de rigidez reducida.
- $\mathbf{Q}$: desplazamientos nodales desconocidos.
- $\mathbf{F}$: cargas nodales equivalentes.

La matriz $\mathbf{K}$ es simétrica y definida positiva.

comprobación de la matriz
La matriz es simétrica:

$$\boxed{\mathbf{K}^T = \mathbf{K}}.$$

Para que CG pueda aplicarse directamente necesitamos además

$$\boxed{\mathbf{Q}^T\mathbf{KQ} > 0 \qquad \forall \mathbf{Q} \neq 0.}$$

En este ejemplo, la matriz tiene diagonal dominante:

$$10 > 2,$$
$$10 > 2 + 2,$$
$$10 > 2 + 2,$$
$$8 > 2.$$

Además, sus autovalores son positivos.
Por tanto,

$$\boxed{\mathbf{K} \text{ es simétrica definida positiva.}}$$

Esto permite utilizar el método del Gradiente Conjugado.

Ejemplo 2: condición inicial

Tomamos

$$\mathbf{Q}_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

El residuo inicial es

$$\mathbf{R}_0 = \mathbf{F} - \mathbf{K}\mathbf{Q}_0 = \mathbf{F} = \begin{bmatrix} 8 \\ 6 \\ 4 \\ 10 \end{bmatrix}$$

y

$$\mathbf{P}_0 = \mathbf{R}_0 = \begin{bmatrix} 8 \\ 6 \\ 4 \\ 10 \end{bmatrix}$$

La primera operación importante es

$$\mathbf{W}_0 = \mathbf{K}\mathbf{P}_0 = \begin{bmatrix} 68 \\ 36 \\ 24 \\ 72 \end{bmatrix}.$$

primera iteración: Calculamos

$$\alpha_0 = \frac{\mathbf{R}_0^T \mathbf{R}_0}{\mathbf{P}_0^T \mathbf{K}\mathbf{P}_0}.$$

El numerador es

$$\mathbf{R}_0^T \mathbf{R}_0 = 8^2 + 6^2 + 4^2 + 10^2 = 216.$$

El denominador es

$$\mathbf{P}_0^T \mathbf{W}_0 = 8(68) + 6(36) + 4(24) + 10(72) = 1360.$$

Por tanto,

$$\alpha_0 = \frac{216}{1360} = 0{,}158824$$

Actualizamos:

$$\mathbf{Q}_1 = \mathbf{Q}_0 + \alpha_0 \mathbf{P}_0 = \begin{bmatrix} 1{,}27059 \\ 0{,}95294 \\ 0{,}63529 \\ 1{,}58824 \end{bmatrix}$$

El nuevo residuo es

$$\mathbf{R}_1 = \mathbf{R}_0 - \alpha_0 \mathbf{W}_0.$$

Por tanto, obtenemos aproximadamente

$$\mathbf{R}_1 = \begin{bmatrix} 8 \\ 6 \\ 4 \\ 10 \end{bmatrix} - 0{,}158824 \begin{bmatrix} 68 \\ 36 \\ 24 \\ 72 \end{bmatrix} = \begin{bmatrix} -2{,}8000 \\ 0{,}28235 \\ 0{,}18824 \\ -1{,}43529 \end{bmatrix}$$

y

$$\|\mathbf{R}_1\| \approx 3{,}15$$

Todavía no hemos alcanzado la convergencia.

Ejemplo 2: actualización del residuo

segunda dirección conjugada
Calculamos

$$\beta_0 = \frac{\mathbf{R}_1^T \mathbf{R}_1}{\mathbf{R}_0^T \mathbf{R}_0} \approx 0{,}04596$$

La nueva dirección es

$$\mathbf{P}_1 = \mathbf{R}_1 + \beta_0 \mathbf{P}_0 \approx \begin{bmatrix} -2{,}4323 \\ 0{,}5581 \\ 0{,}3721 \\ -0{,}9757 \end{bmatrix}$$

Esta dirección no es simplemente el residuo. La combinación con $\mathbf{P}_0$ garantiza la $\mathbf{K}$-conjugación:

$$\boxed{\mathbf{P}_1^T \mathbf{K} \mathbf{P}_0 = 0.}$$

El procedimiento continúa:

$$\mathbf{W}_k = \mathbf{K}\mathbf{P}_k$$

$$\alpha_k = \frac{\mathbf{R}_k^T \mathbf{R}_k}{\mathbf{P}_k^T \mathbf{W}_k}$$

$$\mathbf{Q}_{k+1} = \mathbf{Q}_k + \alpha_k \mathbf{P}_k$$

$$\mathbf{R}_{k+1} = \mathbf{R}_k - \alpha_k \mathbf{W}_k$$

$$\beta_k = \frac{\mathbf{R}_{k+1}^T \mathbf{R}_{k+1}}{\mathbf{R}_k^T \mathbf{R}_k}$$

$$\mathbf{P}_{k+1} = \mathbf{R}_{k+1} + \beta_k \mathbf{P}_k.$$

Como existen cuatro grados de libertad, en aritmética exacta el método converge en a lo sumo cuatro iteraciones.

Ejemplo 2: implementación en Python

```
import numpy as np

K = np.array([
    [10., -2.,  0.,  0.],
    [-2., 10., -2.,  0.],
    [0.,  -2., 10., -2.],
    [0.,   0., -2.,  8.]
])

F = np.array([8., 6., 4., 10.])
Q = np.zeros(4)
R = F - K @ Q
P = R.copy()

tol = 1e-10
max_iter = 100

for k in range(max_iter):
    KP = K @ P
    alpha = (R @ R) / (P @ KP)
    Q = Q + alpha * P
    R_new = R - alpha * KP
    error = np.linalg.norm(R_new)

    print(f"{k+1:2d}  "
          f"||R|| = {error:.6e}")
```

continua $\cdots$

```
    if error < tol:
        break

    beta = (R_new @ R_new) / (R @ R)
    P = R_new + beta * P
    R = R_new

print("\nDesplazamientos:")
print(Q)
```

Verificación con NumPy: Podemos comparar el resultado del método CG con un método directo:

```
# Solucion directa
Q_directo = np.linalg.solve(K, F)

print("CG:")
print(Q)

print("\nMetodo directo:")
print(Q_directo)

print("\nError:")
print(np.linalg.norm(Q - Q_directo))
```

La diferencia debe ser del orden de la tolerancia numérica:

$$\|\mathbf{Q}_{\text{CG}} - \mathbf{Q}_{\text{directo}}\| \approx 0.$$

En un análisis estructural tenemos

$$\boxed{\mathbf{KQ} = \mathbf{F}}.$$

Después de aplicar las condiciones de frontera:

$$\boxed{\mathbf{K}_r\mathbf{Q}_r = \mathbf{F}_r}$$

donde $\mathbf{K}_r$ debe ser simétrica definida positiva para aplicar CG directamente.

El procedimiento general es:

1. Generar la malla.
2. Calcular las matrices elementales $\mathbf{k}^e$.
3. Transformar las matrices, si es necesario.
4. Ensamblar $\mathbf{K}$.
5. Aplicar las condiciones de frontera.
6. Construir $\mathbf{F}$.
7. Resolver
$$\mathbf{K}_r\mathbf{Q}_r = \mathbf{F}_r$$
mediante CG.

Ventaja fundamental: matrices dispersas
En una malla de Elementos Finitos, cada elemento solamente conecta un número reducido de nodos.
Por ello, la matriz global tiene la estructura

$$\mathbf{K} = \begin{bmatrix} \times & \times & 0 & 0 & \cdots \\ \times & \times & \times & 0 & \cdots \\ 0 & \times & \times & \times & \cdots \\ 0 & 0 & \times & \times & \cdots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{bmatrix}.$$

La mayoría de sus componentes son cero.
CG requiere principalmente el producto

$$\boxed{\mathbf{Kp}}.$$

No es necesario calcular

$$\boxed{\mathbf{K}^{-1}}.$$

Idea clave

Para modelos grandes, el producto matriz–vector disperso es mucho más económico que almacenar y factorizar una matriz densa.

Criterio de convergencia

Un criterio habitual es

$$\boxed{\|\mathbf{r}_k\|_2 < \varepsilon}$$

Sin embargo, para comparar problemas de diferentes escalas es conveniente utilizar el residuo relativo:

$$\boxed{\frac{\|\mathbf{r}_k\|_2}{\|\mathbf{b}\|_2} < \varepsilon}$$

Por ejemplo,

$$\varepsilon = 10^{-8}.$$

También podemos controlar el error mediante

$$\boxed{\frac{\|\mathbf{x}_{k+1} - \mathbf{x}_k\|_2}{\|\mathbf{x}_{k+1}\|_2} < \varepsilon.}$$

En Elementos Finitos suele ser preferible controlar el residuo relativo porque está directamente relacionado con el equilibrio del sistema.

¿De qué depende la velocidad de convergencia?
La convergencia de CG está fuertemente relacionada con el número de condición:

$$\boxed{\kappa(\mathbf{A}) = \frac{\lambda_{\text{máx}}}{\lambda_{\text{mín}}}}$$

donde

$$\lambda_{\text{máx}}$$

y

$$\lambda_{\text{mín}}$$

son el mayor y el menor autovalor de $\mathbf{A}$.
En términos generales:

$$\boxed{\kappa(\mathbf{A}) \approx 1 \quad \Longrightarrow \quad \text{convergencia rápida}}$$

mientras que

$$\boxed{\kappa(\mathbf{A}) \gg 1 \quad \Longrightarrow \quad \text{convergencia lenta.}}$$

Por esta razón, en problemas grandes se utilizan precondicionadores.

Gradiente Conjugado Precondicionado

Si el sistema presenta un número de condición elevado,

$$\mathbf{Ax} = \mathbf{b},$$

podemos introducir un precondicionador $\mathbf{M}$:

$$\boxed{\mathbf{M} \approx \mathbf{A}.}$$

En lugar de trabajar directamente con el residuo, resolvemos

$$\boxed{\mathbf{Mz} = \mathbf{r}.}$$

Entonces se utiliza

$$\mathbf{z}_k$$

para construir las nuevas direcciones.
El algoritmo se convierte en el

Gradiente Conjugado Precondicionado (PCG).

Algunos precondicionadores habituales son:

- Jacobi o diagonal.
- Cholesky incompleto (IC).
- ILU.
- Métodos algebraicos multigrid.

CG VERSUS MÉTODOS DIRECTOS

	Método directo	**Gradiente Conjugado**
Ejemplo	Cholesky	CG
Cálculo	Factorización	Iteraciones
Memoria	Puede ser elevada	Generalmente menor
Matrices grandes	Menos conveniente	Muy conveniente
Matriz dispersa	Puede producir relleno	Aprovecha la dispersión
Condición	SPD para Cholesky	SPD para CG

Errores frecuentes al utilizar CG

Error 1

Aplicar CG a una matriz que no es simétrica definida positiva.

Error 2

No aplicar correctamente las condiciones de frontera.

Error 3

Utilizar una tolerancia excesivamente pequeña sin necesidad.

Error 4

No aprovechar la estructura dispersa de $\mathbf{K}$.

Error 5

Calcular explícitamente $\mathbf{K}^{-1}$.

Recomendación

En un código de Elementos Finitos grande, debe evitarse la formación explícita de la inversa y, siempre que sea posible, debe utilizarse almacenamiento disperso.

Función general CG en Python

```
import numpy as np

def gradiente_conjugado(A, b, x0=None, tol=1e-8, max_iter=None):
    n = len(b)

    if x0 is None:
        x = np.zeros(n)
    else:
        x = x0.copy()

    if max_iter is None:
        max_iter = n

    r = b - A @ x
    p = r.copy()
    historial = [np.linalg.norm(r)]

    for k in range(max_iter):
        Ap = A @ p
        alpha = (r @ r) / (p @ Ap)
        x = x + alpha * p
        r_new = r - alpha * Ap

        error = np.linalg.norm(r_new) / np.linalg.norm(b)

        historial.append(np.linalg.norm(r_new))

        if error < tol:
            return x, k + 1, historial

        beta = (r_new @ r_new) / (r @ r)
        p = r_new + beta * p
        r = r_new

    return x, max_iter, historial
```

Aplicación de la función

```
K = np.array([
    [10., -2.,  0.,  0.],
    [-2., 10., -2.,  0.],
    [0.,  -2., 10., -2.],
    [0.,   0., -2.,  8.]
])

F = np.array([8., 6., 4., 10.])

Q, iteraciones, historial = \
    gradiente_conjugado(K, F)

print("Desplazamientos:")
print(Q)

print("Iteraciones:")
print(iteraciones)

print("Residuo␣final:")
print(historial[-1])
```

Pregunta para el estudiante

¿Qué ocurre con el número de iteraciones si aumentamos el número de grados de libertad?

Gráfica de convergencia

-0.3cm

Podemos visualizar la reducción del residuo:

```
import matplotlib.pyplot as plt

plt.semilogy(historial, 'o-')

plt.xlabel('Iteracion')
plt.ylabel(r'$||r_k||_2$')
plt.title('Convergencia del Gradiente Conjugado')

plt.grid(True)
plt.show()
```

La escala logarítmica permite observar claramente la reducción del residuo.

$$\|\mathbf{r}_0\| > \|\mathbf{r}_1\| > \|\mathbf{r}_2\| > \cdots$$

Resumen del método

El sistema de Elementos Finitos es

$$\mathbf{KQ} = \mathbf{F}$$

El método CG requiere:

$$\mathbf{K} = \mathbf{K}^T \quad \mathbf{y} \quad \mathbf{K} > 0.$$

El algoritmo utiliza:

$$\mathbf{r}_k = \mathbf{F} - \mathbf{K}\mathbf{Q}_k$$

$$\alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T \mathbf{K} \mathbf{p}_k}$$

$$\mathbf{Q}_{k+1} = \mathbf{Q}_k + \alpha_k \mathbf{p}_k$$

$$\beta_k = \frac{\mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{r}_k^T \mathbf{r}_k}$$

$$\mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k.$$

Ideas fundamentales para recordar

1. CG es un método iterativo para sistemas lineales.
2. Es especialmente apropiado para matrices simétricas definidas positivas.
3. En Elementos Finitos:

$$\mathbf{KQ} = \mathbf{F}.$$

4. El método evita calcular la inversa de $\mathbf{K}$.
5. Las direcciones de búsqueda son $\mathbf{K}$-conjugadas.
6. La convergencia depende fuertemente del número de condición de la matriz.
7. Para sistemas grandes y mal condicionados, el precondicionamiento puede mejorar considerablemente la convergencia.
8. La estructura dispersa de la matriz global debe aprovecharse en la implementación computacional.

Pregunta de discusión
Considere dos modelos de Elementos Finitos:

$$\mathbf{K}_1\mathbf{Q}_1 = \mathbf{F}_1$$

con

$$\kappa(\mathbf{K}_1) = 20$$

y

$$\mathbf{K}_2\mathbf{Q}_2 = \mathbf{F}_2$$

con

$$\kappa(\mathbf{K}_2) = 10^7.$$

1. ¿Cuál de los dos sistemas debería converger más rápidamente mediante CG?
2. ¿Qué problema puede presentar el segundo sistema?
3. ¿Cómo podría mejorarse su convergencia?
4. ¿Qué papel desempeña un precondicionador?
5. ¿Por qué no es conveniente calcular $\mathbf{K}^{-1}$ explícitamente?

Ejercicio propuesto

Considere el sistema

$$\begin{bmatrix} 6 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 4 \end{bmatrix} \begin{Bmatrix} Q_1 \\ Q_2 \\ Q_3 \end{Bmatrix} = \begin{Bmatrix} 5 \\ 6 \\ 7 \end{Bmatrix}.$$

1. Verifique que la matriz sea simétrica.
2. Verifique que sea definida positiva.
3. Utilice
$$\mathbf{Q}_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$
4. Realice manualmente las primeras dos iteraciones.
5. Implemente CG en Python.
6. Compare el resultado con
`numpy.linalg.solve()`.
7. Grafique
$$\|\mathbf{r}_k\|_2$$
en función de la iteración.

Del ensamblaje a la solución

$$\boxed{\mathbf{KQ} = \mathbf{F}}$$

$$\Downarrow$$

Gradiente Conjugado

$$\Downarrow$$

$$\boxed{\mathbf{Q}}$$

Desplazamientos nodales $\rightarrow$ deformaciones $\rightarrow$ esfuerzos $\rightarrow$ reacciones.

¿Preguntas?

Gradiente Conjugado

Generalized Minimal Residual Method (GMRES)

Es un método iterativo de tipo **Krylov** para resolver sistemas lineales:

$$\mathbf{Ax} = \mathbf{b}$$

donde, en general,

$$\mathbf{A} \neq \mathbf{A}^T.$$

Idea principal

En cada iteración, GMRES busca una aproximación $\mathbf{x}_k$ dentro de un espacio de Krylov que minimiza la norma del residuo:

$$\boxed{\|\mathbf{b} - \mathbf{Ax}_k\|_2}$$

¿Por qué utilizar GMRES?
Muchos problemas de ingeniería conducen a sistemas:

$$\mathbf{Ax} = \mathbf{b}$$

con matrices:

- grandes;
- dispersas;
- no simétricas;
- mal condicionadas;
- provenientes de discretizaciones de EDP.

Ejemplos:

- Mecánica de fluidos.
- Transferencia de calor.
- Elementos finitos.
- Problemas de convección-difusión.
- Dinámica estructural.

Residuo

Partimos de:

$$\mathbf{A}\mathbf{x} = \mathbf{b}.$$

Para una aproximación $\mathbf{x}_k$, definimos el residuo:

$$\mathbf{r}_k = \mathbf{b} - \mathbf{A}\mathbf{x}_k$$

y su norma:

$$\|\mathbf{r}_k\|_2 = \sqrt{\mathbf{r}_k^T \mathbf{r}_k}$$

GMRES intenta encontrar $\mathbf{x}_k$ tal que $\|\mathbf{r}_k\|_2$ sea mínima.

Espacio de Krylov

Sea una aproximación inicial $\mathbf{x}_0$ y:

$$\mathbf{r}_0 = \mathbf{b} - \mathbf{A}\mathbf{x}_0.$$

El espacio de Krylov de dimensión m es:

$$\boxed{\mathcal{K}_m(\mathbf{A}, \mathbf{r}_0) = \text{span}\left\{\mathbf{r}_0, \mathbf{A}\mathbf{r}_0, \mathbf{A}^2\mathbf{r}_0, \ldots, \mathbf{A}^{m-1}\mathbf{r}_0\right\}}$$

La aproximación se construye como:

$$\boxed{\mathbf{x}_m = \mathbf{x}_0 + \mathbf{V}_m \mathbf{y}}$$

donde las columnas de $\mathbf{V}_m$ forman una base ortonormal del espacio de Krylov.

Espacio de Krylov: ¿qué estamos construyendo?

Idea fundamental

El método GMRES no busca directamente la solución $\mathbf{x}$. En cada iteración construye una aproximación dentro de un espacio generado por el residuo inicial y sus imágenes mediante $\mathbf{A}$.

Partimos de:

$$\mathbf{Ax} = \mathbf{b}$$

y elegimos una aproximación inicial:

$$\mathbf{x}_0 = \mathbf{0}.$$

Entonces:

$$\mathbf{r}_0 = \mathbf{b} - \mathbf{Ax}_0 = \mathbf{b}.$$

Pregunta clave

¿Qué información podemos obtener aplicando repetidamente la matriz $\mathbf{A}$ al residuo $\mathbf{r}_0$?

$$\mathbf{r}_0, \quad \mathbf{Ar}_0, \quad \mathbf{A}^2\mathbf{r}_0, \quad \mathbf{A}^3\mathbf{r}_0, \ldots$$

Ejemplo: sistema lineal
Consideremos el sistema:

$$\mathbf{Ax} = \mathbf{b},$$

con

$$\mathbf{A} = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}.$$

Elegimos como aproximación inicial:

$$\mathbf{x}_0 = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^T$$

Por tanto:

$$\mathbf{r}_0 = \mathbf{b} - \mathbf{Ax}_0 = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}.$$

Ahora comenzamos a generar el espacio de Krylov:

$$\mathbf{r}_0, \qquad \mathbf{Ar}_0, \qquad \mathbf{A}^2\mathbf{r}_0, \ldots$$

Primer vector: $\mathbf{r}_0$

Primer vector: $\mathbf{r}_0$
Tenemos directamente:

$$\mathbf{r}_0 = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$

Este vector representa el **error que todavía tenemos** en la ecuación $\mathbf{Ax} = \mathbf{b}$.
Ahora multiplicamos por $\mathbf{A}$:

$$\mathbf{Ar}_0 = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}.$$

Realizando la multiplicación:

$$\mathbf{Ar}_0 = \begin{bmatrix} 2(1) + 1(0) + 0(1) \\ 1(1) + 2(0) + 1(1) \\ 0(1) + 1(0) + 2(1) \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \\ 2 \end{bmatrix}$$

Segundo vector: $\mathbf{A}^2\mathbf{r}_0$
Ya tenemos:

$$\mathbf{Ar}_0 = \begin{bmatrix} 2 \\ 2 \\ 2 \end{bmatrix}.$$

Aplicamos nuevamente la matriz $\mathbf{A}$:

$$\mathbf{A}^2\mathbf{r}_0 = \mathbf{A}(\mathbf{Ar}_0).$$

Por tanto:

$$\mathbf{A}^2\mathbf{r}_0 = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \\ 2 \end{bmatrix} = \begin{bmatrix} 6 \\ 8 \\ 6 \end{bmatrix}$$

Los vectores obtenidos son:

$$\mathbf{r}_0 = \begin{bmatrix}1\\0\\1\end{bmatrix}, \qquad \mathbf{A}\mathbf{r}_0 = \begin{bmatrix}2\\2\\2\end{bmatrix}, \qquad \mathbf{A}^2\mathbf{r}_0 = \begin{bmatrix}6\\8\\6\end{bmatrix}.$$

Por definición:

$$\mathcal{K}_3(\mathbf{A}, \mathbf{r}_0) = \text{span}\left\{\mathbf{r}_0, \mathbf{A}\mathbf{r}_0, \mathbf{A}^2\mathbf{r}_0\right\}.$$

Es decir:

$$\mathcal{K}_3 = \text{span}\left\{\begin{bmatrix}1\\0\\1\end{bmatrix}, \begin{bmatrix}2\\2\\2\end{bmatrix}, \begin{bmatrix}6\\8\\6\end{bmatrix}\right\}$$

Interpretación

El espacio de Krylov contiene todas las combinaciones lineales posibles de estos vectores.

¿Qué significa "span"?

Si

$$\mathcal{K}_3 = \text{span}\{\mathbf{r}_0, \mathbf{A}\mathbf{r}_0, \mathbf{A}^2\mathbf{r}_0\},$$

entonces cualquier vector $\mathbf{v}$ perteneciente a $\mathcal{K}_3$ puede escribirse como:

$$\boxed{\mathbf{v} = \alpha_1\mathbf{r}_0 + \alpha_2\mathbf{A}\mathbf{r}_0 + \alpha_3\mathbf{A}^2\mathbf{r}_0}$$

Por ejemplo:

$$\mathbf{v} = 2\mathbf{r}_0 - \mathbf{A}\mathbf{r}_0 + \frac{1}{2}\mathbf{A}^2\mathbf{r}_0.$$

Entonces:

$$\mathbf{v} = 2\begin{bmatrix}1\\0\\1\end{bmatrix} - \begin{bmatrix}2\\2\\2\end{bmatrix} + \frac{1}{2}\begin{bmatrix}6\\8\\6\end{bmatrix} = \begin{bmatrix}3\\2\\3\end{bmatrix}$$

Aquí aparece la idea fundamental de GMRES. En lugar de buscar $\mathbf{x}$ en todo $\mathbb{R}^n$, buscamos una aproximación dentro de:

$$\boxed{\mathbf{x}_m = \mathbf{x}_0 + \mathbf{v}, \qquad \mathbf{v} \in \mathcal{K}_m}$$

Como $\mathbf{v}$ pertenece al espacio de Krylov:

$$\mathbf{v} = \mathbf{V}_m \mathbf{y}.$$

Por tanto:

$$\boxed{\mathbf{x}_m = \mathbf{x}_0 + \mathbf{V}_m \mathbf{y}}$$

donde:

$$\mathbf{V}_m = \begin{bmatrix} \mathbf{v}_1 & \mathbf{v}_2 & \cdots & \mathbf{v}_m \end{bmatrix}$$

contiene una **base ortonormal** del espacio de Krylov.

¿Por qué necesitamos una base ortonormal?
Los vectores

$$\mathbf{r}_0, \quad \mathbf{A}\mathbf{r}_0, \quad \mathbf{A}^2\mathbf{r}_0, \ldots$$

no necesariamente son ortogonales. Además, pueden tener magnitudes muy diferentes. GMRES utiliza el proceso de **Arnoldi** para transformar:

$$\left\{\mathbf{r}_0, \mathbf{A}\mathbf{r}_0, \mathbf{A}^2\mathbf{r}_0, \ldots\right\}$$

en una base ortonormal:

$$\mathbf{V}_m = [\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_m]$$

tal que:

$$\boxed{\mathbf{V}_m^T \mathbf{V}_m = \mathbf{I}}$$

Idea

Krylov genera el espacio; Arnoldi construye una base ortonormal para trabajar numéricamente con ese espacio.

Visualización conceptual

Generación del espacio

$$\mathbf{r}_0$$
$$\downarrow \mathbf{A}$$
$$\mathbf{A}\mathbf{r}_0$$
$$\downarrow \mathbf{A}$$
$$\mathbf{A}^2\mathbf{r}_0$$
$$\downarrow \mathbf{A}$$
$$\mathbf{A}^3\mathbf{r}_0$$

Estos vectores generan:

$$\mathcal{K}_m(\mathbf{A}, \mathbf{r}_0).$$

Ortogonalización de Arnoldi

$$\begin{aligned} \mathbf{r}_0 &\longrightarrow \mathbf{v}_1 \\ \mathbf{A}\mathbf{r}_0 &\longrightarrow \mathbf{v}_2 \\ \mathbf{A}^2\mathbf{r}_0 &\longrightarrow \mathbf{v}_3 \end{aligned}$$

Finalmente:

$$\mathbf{V}_m = \begin{bmatrix} | & | & & | \\ \mathbf{v}_1 & \mathbf{v}_2 & \cdots & \mathbf{v}_m \\ | & | & & | \end{bmatrix}$$

La idea en una sola imagen matemática

$$\begin{array}{l} \mathbf{r}_0 \\ \downarrow \mathbf{A} \\ \mathbf{A}\mathbf{r}_0 \\ \downarrow \mathbf{A} \\ \mathbf{A}^2\mathbf{r}_0 \\ \downarrow \mathbf{A} \\ \vdots \end{array} \quad \Longrightarrow \quad \mathcal{K}_m(\mathbf{A}, \mathbf{r}_0)$$

Arnoldi transforma estos vectores en:

$$\mathbf{V}_m = [\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_m]$$

y GMRES busca:

$$\mathbf{x}_m = \mathbf{x}_0 + \mathbf{V}_m \mathbf{y}$$

escogiendo $\mathbf{y}$ de manera que el residuo sea mínimo:

$$\mathbf{y} = \arg\min_{\mathbf{y}} \|\mathbf{b} - \mathbf{A}(\mathbf{x}_0 + \mathbf{V}_m \mathbf{y})\|$$

Resumen

Krylov genera el espacio → Arnoldi genera la base ortonormal → GMRES busca la mejor aproximación.

Algoritmo de Arnoldi

El algoritmo de Arnoldi genera una base ortonormal:

$$\mathbf{V}_m = \begin{bmatrix} \mathbf{v}_1 & \mathbf{v}_2 & \cdots & \mathbf{v}_m \end{bmatrix}.$$

Partimos de:

$$\mathbf{v}_1 = \frac{\mathbf{r}_0}{\|\mathbf{r}_0\|_2}.$$

Para $j = 1, \ldots, m$:

$$\mathbf{w} = \mathbf{A}\mathbf{v}_j.$$

Luego:

$$h_{ij} = \mathbf{v}_i^T \mathbf{w}, \qquad i = 1, \ldots, j.$$

Ortogonalizamos:

$$\mathbf{w} = \mathbf{w} - \sum_{i=1}^{j} h_{ij}\mathbf{v}_i.$$

Finalmente:

$$h_{j+1,j} = \|\mathbf{w}\|_2$$

y

$$\boxed{\mathbf{v}_{j+1} = \frac{\mathbf{w}}{h_{j+1,j}}}$$

Relación de Arnoldi
El proceso anterior produce:

$$\boxed{\mathbf{A}\mathbf{V}_m = \mathbf{V}_{m+1}\overline{\mathbf{H}}_m}$$

donde:

$$\mathbf{V}_{m+1} = \begin{bmatrix} \mathbf{v}_1 & \cdots & \mathbf{v}_{m+1} \end{bmatrix}$$

y $\overline{\mathbf{H}}_m$ es una matriz de Hessenberg superior:

$$\overline{\mathbf{H}}_m = \begin{bmatrix} h_{11} & h_{12} & \cdots & h_{1m} \\ h_{21} & h_{22} & \cdots & h_{2m} \\ 0 & h_{32} & \cdots & h_{3m} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & h_{m+1,m-1} & h_{m+1,m} \end{bmatrix}.$$

Reducción del problema

Tenemos:

$$\mathbf{x}_m = \mathbf{x}_0 + \mathbf{V}_m \mathbf{y}.$$

El residuo es:

$$\mathbf{r}_m = \mathbf{b} - \mathbf{A}\left(\mathbf{x}_0 + \mathbf{V}_m \mathbf{y}\right).$$

Como:

$$\mathbf{r}_0 = \mathbf{b} - \mathbf{A}\mathbf{x}_0,$$

entonces:

$$\mathbf{r}_m = \mathbf{r}_0 - \mathbf{A}\mathbf{V}_m \mathbf{y}.$$

Utilizando Arnoldi:

$$\mathbf{r}_m = \mathbf{V}_{m+1}\left(\beta \mathbf{e}_1 - \overline{\mathbf{H}}_m \mathbf{y}\right)$$

donde:

$$\boxed{\beta = \|\mathbf{r}_0\|_2}$$

Problema de mínimos cuadrados
Como $\mathbf{V}_{m+1}$ tiene columnas ortonormales:

$$\|\mathbf{r}_m\|_2 = \left\|\beta \mathbf{e}_1 - \overline{\mathbf{H}}_m \mathbf{y}\right\|_2.$$

Por tanto, GMRES resuelve el problema:

$$\boxed{\min_{\mathbf{y}} \left\|\beta \mathbf{e}_1 - \overline{\mathbf{H}}_m \mathbf{y}\right\|_2}$$

Una vez obtenida $\mathbf{y}$:

$$\boxed{\mathbf{x}_m = \mathbf{x}_0 + \mathbf{V}_m \mathbf{y}}$$

Resultado fundamental

Un sistema lineal grande se transforma en un problema de mínimos cuadrados de dimensión mucho menor.

Rotaciones de Givens

Para resolver eficientemente el problema de mínimos cuadrados se utilizan rotaciones de Givens.
Una rotación tiene la forma:

$$\mathbf{G} = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}$$

con:

$$c = \frac{a}{\sqrt{a^2 + b^2}}, \qquad s = \frac{b}{\sqrt{a^2 + b^2}}.$$

La rotación permite transformar:

$$\begin{bmatrix} a \\ b \end{bmatrix} \rightarrow \begin{bmatrix} r \\ 0 \end{bmatrix}.$$

De esta manera, GMRES puede actualizar el residuo sin resolver nuevamente todo el problema de mínimos cuadrados.

Algoritmo GMRES: inicialización

Dados:

$$\mathbf{A}, \quad \mathbf{b}, \quad \mathbf{x}_0,$$

calcular:

$$\mathbf{r}_0 = \mathbf{b} - \mathbf{A}\mathbf{x}_0$$

y:

$$\beta = \|\mathbf{r}_0\|_2.$$

Si:

$$\beta < tol,$$

entonces $\mathbf{x}_0$ ya es una solución suficientemente precisa.
En caso contrario:

$$\boxed{\mathbf{v}_1 = \frac{\mathbf{r}_0}{\beta}}$$

Algoritmo GMRES: Arnoldi
Para:

$$j = 1, 2, \ldots, m$$

calcular:

$$\mathbf{w} = \mathbf{A}\mathbf{v}_j.$$

Para $i = 1, \ldots, j$:

$$h_{ij} = \mathbf{v}_i^T \mathbf{w}$$

$$\mathbf{w} = \mathbf{w} - h_{ij}\mathbf{v}_i.$$

Después:

$$h_{j+1,j} = \|\mathbf{w}\|_2.$$

Si $h_{j+1,j} \neq 0$:

$$\mathbf{v}_{j+1} = \frac{\mathbf{w}}{h_{j+1,j}}.$$

Algoritmo GMRES: solución

Construimos:

$$\overline{\mathbf{H}}_j .$$

Resolvemos:

$$\boxed{\mathbf{y}_j = \arg\min_{\mathbf{y}} \left\| \beta \mathbf{e}_1 - \overline{\mathbf{H}}_j \mathbf{y} \right\|_2}$$

y actualizamos:

$$\boxed{\mathbf{x}_j = \mathbf{x}_0 + \mathbf{V}_j \mathbf{y}_j}$$

Calculamos:

$$\mathbf{r}_j = \mathbf{b} - \mathbf{A}\mathbf{x}_j .$$

Si:

$$\boxed{\|\mathbf{r}_j\|_2 < tol}$$

el algoritmo termina.

Pseudocódigo de GMRES

1. $\mathbf{r}_0 = \mathbf{b} - \mathbf{A}\mathbf{x}_0$
2. $\beta = \|\mathbf{r}_0\|_2$
3. $\mathbf{v}_1 = \mathbf{r}_0 / \beta$
4. Para $j = 1, \ldots, m$:
   - $\mathbf{w} = \mathbf{A}\mathbf{v}_j$
   - Ortogonalizar $\mathbf{w}$
   - Construir $\overline{\mathbf{H}}_j$
   - Resolver el problema de mínimos cuadrados
   - Calcular $\mathbf{x}_j$
   - Evaluar el residuo
5. Si el residuo es menor que la tolerancia:

   STOP

Ejemplo 1: sistema 3×3

Consideremos:

$$\mathbf{A} = \begin{bmatrix} 4 & 1 & 0 \\ 2 & 3 & 1 \\ 0 & 1 & 2 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}.$$

Resolver:

$$\mathbf{A}\mathbf{x} = \mathbf{b}.$$

Tomamos:

$$\mathbf{x}_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

Entonces:

$$\mathbf{r}_0 = \mathbf{b} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}.$$

$$\beta = \|\mathbf{r}_0\|_2 = \sqrt{14}.$$

primer vector de Krylov

Tenemos:

$$\mathbf{v}_1 = \frac{1}{\sqrt{14}} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}.$$

Calculamos:

$$\mathbf{w} = \mathbf{A}\mathbf{v}_1.$$

Por tanto:

$$\mathbf{w} = \frac{1}{\sqrt{14}} \begin{bmatrix} 6 \\ 11 \\ 8 \end{bmatrix}.$$

Primer coeficiente:

$$h_{11} = \mathbf{v}_1^T \mathbf{w}.$$

Entonces:

$$h_{11} = \frac{52}{14} = \frac{26}{7}.$$

Ejemplo 1:

segundo vector
Ortogonalizamos:

$$\mathbf{w} = \mathbf{A}\mathbf{v}_1 - h_{11}\mathbf{v}_1.$$

Luego:

$$h_{21} = \|\mathbf{w}\|_2.$$

Finalmente:

$$\mathbf{v}_2 = \frac{\mathbf{w}}{h_{21}}.$$

El procedimiento continúa de forma análoga para obtener:

$$\mathbf{v}_3.$$

Con estos vectores construimos:

$$\mathbf{V}_3 = [\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3].$$

sistema reducido
Después del proceso de Arnoldi obtenemos:

$$\mathbf{A}\mathbf{V}_m = \mathbf{V}_{m+1}\overline{\mathbf{H}}_m.$$

GMRES resuelve:

$$\min_{\mathbf{y}} \left\| \sqrt{14}\mathbf{e}_1 - \overline{\mathbf{H}}_m\mathbf{y} \right\|_2.$$

Finalmente:

$$\boxed{\mathbf{x}_m = \mathbf{V}_m\mathbf{y}}$$

porque $\mathbf{x}_0 = 0$.
solución exacta
La solución del sistema es:

$$\mathbf{x} = \begin{bmatrix} -1/7 \\ 11/7 \\ 4/7 \end{bmatrix}$$

es decir:

$$\mathbf{x} \approx \begin{bmatrix} -0{,}142857 \\ 1{,}571429 \\ 0{,}571429 \end{bmatrix}$$

GMRES converge en un número de iteraciones no mayor que $n = 3$ en aritmética exacta.

Ejemplo 2: sistema no simétrico

Consideremos:

$$\mathbf{A} = \begin{bmatrix} 4 & 1 & 0 & 0 \\ -1 & 4 & 1 & 0 \\ 0 & -1 & 4 & 1 \\ 0 & 0 & -1 & 4 \end{bmatrix}$$

y:

$$\mathbf{b} = \begin{bmatrix} 5 \\ 4 \\ 4 \\ 5 \end{bmatrix}.$$

Resolver:

$$\mathbf{Ax} = \mathbf{b}.$$

Tomamos:

$$\mathbf{x}_0 = \mathbf{0}.$$

Este sistema es:

$$\boxed{\mathbf{A} \neq \mathbf{A}^T}$$

por lo que GMRES resulta especialmente apropiado.

primer paso
El residuo inicial es:

$$\mathbf{r}_0 = \mathbf{b}.$$

Su norma:

$$\beta = \sqrt{5^2 + 4^2 + 4^2 + 5^2} = \sqrt{82}.$$

Por tanto:

$$\mathbf{v}_1 = \frac{1}{\sqrt{82}} \begin{bmatrix} 5 \\ 4 \\ 4 \\ 5 \end{bmatrix}.$$

Calculamos:

$$\mathbf{w} = \mathbf{A}\mathbf{v}_1.$$

Después aplicamos el proceso de ortogonalización de Arnoldi.

Ejemplo 2: construcción de Hessenberg

En cada iteración:

$$\mathbf{w} = \mathbf{A}\mathbf{v}_j.$$

Calculamos:

$$h_{ij} = \mathbf{v}_i^T \mathbf{w}.$$

Y:

$$\mathbf{w} \leftarrow \mathbf{w} - h_{ij}\mathbf{v}_i.$$

Finalmente:

$$h_{j+1,j} = \|\mathbf{w}\|_2.$$

Así construimos:

$$\overline{\mathbf{H}}_m.$$

minimización
GMRES obtiene:

$$\mathbf{y}_m = \arg\min_{\mathbf{y}} \left\| \sqrt{82}\mathbf{e}_1 - \overline{\mathbf{H}}_m \mathbf{y} \right\|_2.$$

Después:

$$\mathbf{x}_m = \mathbf{V}_m \mathbf{y}_m.$$

El proceso se detiene cuando:

$$\frac{\|\mathbf{b} - \mathbf{A}\mathbf{x}_m\|_2}{\|\mathbf{b}\|_2} < 10^{-10}.$$

]solución La solución exacta se puede obtener mediante un método directo:

$$\boxed{\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}}$$

o, preferentemente en Python:

```
x_exact = np.linalg.solve(A, b)
```

GMRES debe producir una solución numéricamente equivalente.

Python: implementación básica, Arnoldi

Utilizaremos:

- NumPy para álgebra lineal.
- Matplotlib para las gráficas.
- SciPy para comparar con una implementación profesional.

```
import numpy as np
import matplotlib.pyplot as plt
```

La implementación siguiente utiliza explícitamente:

- espacio de Krylov;
- Arnoldi;
- mínimos cuadrados;
- cálculo del residuo.

Python: GMRES desde cero

```
import numpy as np

def gmres(A, b, x0=None, tol=1e-10, restart=None):
    n = len(b)

    if x0 is None:
        x0 = np.zeros(n)

    if restart is None:
        restart = n

    x = x0.copy()
    r = b - A @ x
    beta = np.linalg.norm(r)
    residuals = [beta]

    if beta < tol:
        return x, residuals

    V = np.zeros((n, restart + 1))
    H = np.zeros((restart + 1, restart))
    V[:, 0] = r / beta

    for j in range(restart):
        w = A @ V[:, j]

        for i in range(j + 1):
            H[i, j] = np.dot(V[:, i], w)
            w = w - H[i, j] * V[:, i]

        H[j + 1, j] = np.linalg.norm(w)

        if H[j + 1, j] < 1e-14:
            break

        V[:, j + 1] = w / H[j + 1, j]
```

Python: mínimos cuadrados

```
        e1 = np.zeros(j + 2)
        e1[0] = beta
        Hj = H[:j+2, :j+1]

        y, _, _, _ = np.linalg.lstsq(
            Hj, e1, rcond=None
        )

        x_new = x + V[:, :j+1] @ y
        r = b - A @ x_new
        residual = np.linalg.norm(r)
        residuals.append(residual)

        if residual < tol:
            return x_new, residuals

    return x_new, residuals
```

Observación sobre la implementación

La implementación anterior es didáctica:

$$\boxed{\text{Arnoldi} \to \text{Hessenberg} \to \text{mínimos cuadrados} \to \mathbf{x}_k}$$

En una implementación de alto rendimiento se utilizan:

- rotaciones de Givens;
- matrices dispersas;
- precondicionadores;
- GMRES con reinicio.

Python: Ejemplo 1

```
A1 = np.array([
    [4., 1., 0.],
    [2., 3., 1.],
    [0., 1., 2.]
])

b1 = np.array([1., 2., 3.])
x1, res1 = gmres(A1, b1, tol=1e-10)

print("GMRES:")
print(x1)

print("Exacta:")
print(np.linalg.solve(A1, b1))

print("Residuo:")
print(np.linalg.norm(b1 - A1 @ x1))
```

Python: gráfica del residuo

```
plt.figure()
plt.semilogy(range(len(res1)), res1, 'o-')

plt.xlabel("Iteracion")
plt.ylabel(r"$||r_k||_2$")
plt.title("GMRES - Ejemplo 1")
plt.grid(True)

plt.show()
```

Python: Ejemplo 2

```
A2 = np.array([
    [4., 1., 0., 0.],
    [-1., 4., 1., 0.],
    [0., -1., 4., 1.],
    [0., 0., -1., 4.]
])

b2 = np.array([5., 4., 4., 5.])

x2, res2 = gmres(A2, b2, tol=1e-10)

print("GMRES:")
print(x2)

print("Exacta:")
print(np.linalg.solve(A2, b2))

print("Residuo:")
print(np.linalg.norm(b2 - A2 @ x2))
```

Python: gráfica del Ejemplo 2

```
plt.figure()
plt.semilogy(range(len(res2)), res2, 's-')
plt.xlabel("Iteracion")
plt.ylabel(r"$||r_k||_2$")
plt.title("GMRES - Ejemplo 2")

plt.grid(True)
plt.show()
```

GMRES utilizando SciPy

Para problemas reales podemos utilizar:

```
from scipy.sparse.linalg import gmres
```

Ejemplo:

```
x, info = gmres(A2, b2, rtol=1e-10, atol=1e-12)
print(x)
print(info)
```

La implementación de SciPy está diseñada para resolver problemas numéricos de manera más eficiente que la implementación didáctica.

Problema del almacenamiento

GMRES completo almacena:
$\mathbf{V}_m = [\mathbf{v}_1, \ldots, \mathbf{v}_m]$ Para n muy grande, el almacenamiento puede convertirse en un problema. Aproximadamente:

$$\text{Memoria} \sim O(nm)$$

Por esta razón se utiliza:

$$\text{GMRES}(m)$$

o GMRES con reinicio.

GMRES con reinicio

El procedimiento es:

1. Ejecutar GMRES durante m iteraciones.
2. Obtener $\mathbf{x}_m$.
3. Calcular:
$$\mathbf{r}_m = \mathbf{b} - \mathbf{A}\mathbf{x}_m.$$
4. Reiniciar:
$$\mathbf{x}_0 \leftarrow \mathbf{x}_m.$$
5. Construir nuevamente el espacio de Krylov.
$$\text{GMRES}(m)$$

permite controlar el consumo de memoria.

Precondicionamiento
La convergencia puede mejorar considerablemente utilizando un precondicionador:

$$\boxed{\mathbf{M}^{-1}\mathbf{A}\mathbf{x} = \mathbf{M}^{-1}\mathbf{b}}$$

donde:

$$\mathbf{M} \approx \mathbf{A}.$$

El objetivo es que:

$$\mathbf{M}^{-1}\mathbf{A}$$

tenga mejores propiedades espectrales que $\mathbf{A}$.
Ejemplos:

- Jacobi.
- Gauss–Seidel.
- ILU.
- Factorización incompleta.

GMRES frente a otros métodos

Método	Simétrica	SPD	No simétrica
Jacobi	Sí/No	Sí/No	Sí/No
Gauss–Seidel	Sí/No	Sí/No	Sí/No
CG	Sí	Sí	No
BiCGSTAB	No	No	Sí
GMRES	No	No	**Sí**

CG:

$$\mathbf{A} = \mathbf{A}^T, \qquad \mathbf{A} \succ 0 \quad \textbf{(definida positiva)}$$

GMRES:

$\mathbf{A}$ puede ser no simétrica

Aplicaciones en ingeniería

GMRES puede utilizarse en:

- **Mecánica de fluidos:**

$$\mathbf{Au} = \mathbf{b}$$

- **Transferencia de calor**.
- **Convección–difusión**.
- **Elementos finitos**.
- **Métodos de diferencias finitas**.
- **Problemas multifísicos**.

Especialmente cuando:

$$n \gg 1$$

y la matriz es dispersa y no simétrica.

Resumen del método GMRES

GMRES resuelve:

$$\mathbf{Ax} = \mathbf{b}$$

mediante:

$$\boxed{\text{Espacio de Krylov} \rightarrow \text{Arnoldi} \rightarrow \text{Hessenberg} \rightarrow \text{mínimos cuadrados}}$$

La aproximación es:

$$\mathbf{x}_m = \mathbf{x}_0 + \mathbf{V}_m \mathbf{y}_m$$

donde:

$$\boxed{\mathbf{y}_m = \arg\min_{\mathbf{y}} \left\| \beta \mathbf{e}_1 - \overline{\mathbf{H}}_m \mathbf{y} \right\|_2}$$

1. GMRES es un método iterativo de Krylov.
2. No requiere que **A** sea simétrica.
3. Minimiza directamente la norma del residuo.
4. Arnoldi construye una base ortonormal.
5. El problema grande se reduce a un problema pequeño de mínimos cuadrados.
6. Las rotaciones de Givens permiten implementar eficientemente la minimización.
7. Para problemas grandes se utiliza GMRES con reinicio.
8. El precondicionamiento puede acelerar significativamente la convergencia.

Mensaje final

GMRES

"No necesitamos invertir la matriz."

Buscamos la solución dentro de un espacio de Krylov

La eficiencia está en convertir un problema grande en una sucesión de problemas pequeños.

¿Preguntas?

Método GMRES

Gracias por su atención

¿Preguntas?

References